Journal of Manufacturing Systems 85 (2026) 642–661

Contents lists available at ScienceDirect

Journal of Manufacturing Systems

journal homepage: www.elsevier.com/locate/jmansys

Technical paper

LLM-driven discrete-event simulation: A generative AI framework for automated model generation, adaptation, and evaluation in manufacturing☆

Thomas Schmitt [a,b,*], Nils Gegenmantel [a,b], Matías Urenda Moris [b], Pär Mårtensson [a], Kaveh Amouzgar [b,c]

[a] Scania CV AB, Global Industrial Development, Södertälje, 151 38, Sweden
[b] Uppsala University, Department of Civil and Industrial Engineering, Division of Industrial Engineering & Management, Uppsala, 752 37, Sweden
[c] University of Skövde, School of Engineering Science, 541 28, Skövde, Sweden

ARTICLE INFO

Keywords:
Discrete-event simulation
Generative artificial intelligence
Large language models
Simulation automation
Manufacturing system analysis
Decision support

ABSTRACT

This paper presents an end-to-end generative artificial intelligence (Gen-AI) framework for automating the generation, adaptation, and evaluation of discrete-event simulation (DES) models in manufacturing. The approach integrates multiple large language models (LLMs) with a structured blueprint model and targeted human-in-the-loop controls to create executable simulation models from heterogeneous production data, implement targeted modifications, and interpret simulation outcomes. The workflow incorporates prompt engineering, zero- and one-shot implementations, and evaluator–optimizer loops. 21 experimental runs on two industrial case studies from a Swedish automotive manufacturer demonstrate that LLMs can support DES model generation and scenario exploration through a hybrid approach combining automation with human oversight. The results underline both the potential and current limitations of LLM-driven simulation, particularly regarding output consistency and generalizability. Future research should extend the method to more complex manufacturing systems and investigate the role of emerging autonomous Gen-AI tools in simulation-based decision support.

1. Introduction

Manufacturing industries are under relentless pressure due to increasing competition, supply chain uncertainties, volatile energy prices, and climate ambitions. In this environment, productivity and energy efficiency are essential for competitiveness. For decades, manufacturing engineering has adapted and developed tools to design and manage production systems with maximum efficiency [1]. Among these, discrete-event simulation (DES) stands out as a powerful method for analyzing material flows, measuring capacity, locating bottlenecks (BTs), and testing process improvements using design of experiments [2] and optimization [3].

Yet, DES modeling remains time- and resource-intensive. Building and adapting models demands scarce expertise in simulation software and advanced data analysis, still rare skills in the manufacturing industry. Moreover, models need to be continuously updated to reflect dynamic production environments, adding further complexity to their practical application.

Here, recent advances in generative artificial intelligence (Gen-AI), particularly the development and deployment of large language models (LLMs), offer opportunities to automate and scale complex tasks in manufacturing [4,5]. Given user prompts and input data, LLM agents can generate text, code, and structured data, significantly reducing the manual work of software programming and data analysis [6]. LLMs' capabilities in coding, visual understanding, scientific understanding, and mathematics are constantly evolving, with coding being the area where they currently perform most reliably [7]. In practice, this means that once hours-long manual programming and handling of data can now be achieved much faster, by integrating LLM agents into company workflows to generate and execute code.

These capabilities are still in their infancy. The transformer architecture, which underpins today's LLMs, was first introduced in 2017 [8]. In November 2022, chatbot tools like ChatGPT from the company OpenAI reached the general public. Unsurprisingly, applications of Gen-AI in manufacturing, especially in DES, remain rare, with limited understanding of their practical value, benefits, and constraints.

☆ This article is part of a Special issue entitled: 'LLMs for Smart Mfg' published in Journal of Manufacturing Systems.

* Corresponding author at: Scania CV AB, Global Industrial Development, Södertälje, 151 38, Sweden.
E-mail address: thomas.schmitt@scania.com (T. Schmitt).

https://doi.org/10.1016/j.jmsy.2026.02.015
Received 15 September 2025; Received in revised form 27 January 2026; Accepted 13 February 2026
Available online 23 February 2026

Abbreviation	Description
API	Application programming interface
BT	Bottleneck
CPD	Critical process definition
CT	Cycle time
DES	Discrete-event simulation
DT	Digital twin
Gen-AI	Generative artificial intelligence
GPT	Generative pre-trained transformer
GUI	Graphical user interface
JSON	JavaScript Object Notation
KPI	Key performance indicator
LLM	Large language model
MES	Manufacturing execution system
MCP	Model context protocol
MTTR	Mean-time-to-repair
NLI	Natural language interface
NLP	Natural language processing
SEC	Specific energy consumption
TH	Throughput
WIP	Work-in-process

To address this, this study investigates how LLMs can be used to automate and adapt DES modeling for capacity and energy efficiency assessments in manufacturing. Specifically, this study aims to develop a comprehensive LLM-driven workflow that can automatically generate, modify, and evaluate simulation models, thereby expanding the exploration and analysis of industrial process improvements and providing effective decision support.

To achieve this, the paper addresses two research questions (RQs):

RQ1: How can LLMs be applied to generate, adapt, and evaluate simulations of manufacturing systems?

RQ2: What are the main challenges and benefits associated with implementing LLM-driven simulations?

By combining the strengths of LLM agents in natural language processing (NLP) and code generation with the analytical power of simulations, this study introduces a workflow validated in two real-world case studies at a large Swedish automotive manufacturer. The results highlight both the technical feasibility and managerial relevance of integrating LLM-driven simulations into manufacturing practice.

2. Background

To effectively apply Gen-AI for simulation modeling in manufacturing, it is crucial to understand both the underlying Gen-AI technologies and the core concepts of manufacturing process simulation. This section explains terms and concepts and by that establishes the foundations for integrating these methodologies.

2.1. Generative artificial intelligence: Core concepts

Gen-AI is a subset of artificial intelligence, leveraging transformer-based deep learning models, particularly LLMs [6]. LLMs are deep learning models trained to understand and generate different types of data. The most prominent type of LLMs are generative pre-trained transformers (GPTs). The LLMs put words into tokens, looking for associated terms to these tokens in its large training data. Then, they produce vectors of tokens which express its similarity to related terms, which are used to cluster related tokens together [6]. Then, a tool called transformer comes in, using a self-attention mechanism to analyze the entire text at once to understand relations and assign probabilities between tokens [8]. These probabilities are used to predict the most fitting sequence.

LLMs extend beyond text generation, producing software code and structured data, and more recently images and videos through what are known as large multi-modal models [9,10]. The interest by industry in LLMs are due to its proclaimed benefits in increasing productivity by mostly generating text and code; tests have shown that LLMs often excel human-level performance in these tasks [11]. Neither the text- nor the code-generating capabilities of many LLMs are bound to specific (programming) languages, thus can significantly enhance the efficiency of tasks such as software programming (e.g., [12]) and data analysis (e.g., [6]). This makes them particularly valuable for process optimizations using simulations [4].

LLMs are the technology behind OpenAI's GPT-5, Anthropic's Claude, or Google's Gemini models. OpenAI, for example, built its models into the chat-bot ChatGPT, a user interface connected to different LLMs that were trained on large amounts of data scraped from the internet, and fine-tuned for generating text and code in response to user queries. Lately, AI companies such as OpenAI directly offer application programming interfaces (APIs), to implement its models into company-code workflows.

The AI company Anthropic distinguishes between two types of *agentic systems*: *workflows* and *agents* [13]. In their definition, workflows are predefined code paths that leverage LLMs to generate outputs used in subsequent deterministic code executions. This approach is well-suited to applications requiring high output consistency. In contrast, by agents they refer to autonomous decision-making systems in which LLMs dynamically interact with other tools or environments, for example via model context protocol (MCP) [14]. To provide a manufacturing analogy, a workflow can be compared to an automated guided vehicle that follows pre-defined paths using magnetic stripes on the factory floor, while an agent can be compared to an autonomous mobile robot that navigates autonomously and adapts decisions based on its embedded sensing devices. In this paper, we focus on building an effective LLM-driven workflow within a rather narrow code path, as in the way described above, however, with agents we do not refer to a fully autonomous system but instead refer to individual LLM calls integrated into the workflow.

2.2. Manufacturing system simulation

One of the most prominent way of modeling, analyzing, and optimizing production processes in industry is using DES models. Simulations replicate the real manufacturing system's behavior, and allow to test scenarios and their effect on objectives such as productivity and energy efficiency [2,15]. Designing DES models requires a thorough understanding of the manufacturing system, data pre-processing, data analysis, and simulation software. Once a model is built and validated, it allows for detailed analysis of the current and future system behaviors, its BTs, and the testing of improvement strategies.

Manufacturing simulation typically follows a structured, systematic approach consisting of: (i) case definition in which the use case and objectives are defined, (ii) current state analysis in which the initial model is built reflecting the real-world production operations, (iii) future state predictions in which the model is adjusted and parameters and/or structural changes are tested for system improvements, and (iv) decision-support provided to management [3].

Building on these concepts, the following literature review examines prior work in simulation modeling, the evolution of Gen-AI applications in manufacturing, and recent studies exploring the integration of LLMs with simulations.

3. Related works

3.1. Automation approaches to simulation model and digital twin developments

Over the past two decades, researchers have proposed a variety of methods to reduce the effort involved in simulation model development within manufacturing systems. Early approaches relied heavily on

rule-based modeling and structured templates [16–18]. These methods typically required detailed domain knowledge encoded in component libraries or parametric templates.

To improve accessibility and usability, especially for non-programmers, interface-facilitated solutions were introduced. These include graphical model builders and spreadsheet-based tools, which enable users to define simulation logic and parameters through visual or tabular interfaces [19,20]. While these tools reduce the technical barrier, they still require manual configuration and do not fully automate model generation.

More recently, the emergence of Industry 4.0 has driven the development of data-driven simulation approaches. These methods leverage event logs, sensor data, and process mining to automate the creation of simulation models and digital twins (DTs), offering the potential for real-time synchronization between virtual models and physical systems [21–23]. Although promising, these techniques typically depend on highly structured data and require expert intervention to ensure model accuracy and domain-specific adaptability. In the context of DES, the diversity and granularity of input data further complicate the development of fully automated, end-to-end DT pipelines [24]. Data collection itself represents a considerable share of the overall effort; Skoogh et al. [25] report that gathering and preparing input data can account for roughly one third of the total project time in simulation studies.

Cimino et al. [26] conducted a systematic review of automatic simulation model generation (ASMG) strategies in manufacturing, outlining (i) data-centric modeling that leverages raw production data for, among others, DTs, (ii) algorithmic approaches using machine learning techniques to scale the generation of simulation models, (iii) knowledge-based approaches using templates, structures, and ontologies, and (iv) interface-driven approaches including graphical user interfaces (GUIs) and natural language interfaces (NLIs). While their study demonstrates the growing maturity of ASMG methodologies, they emphasize that the integration of LLMs with NLIs remains in its infancy, representing a promising but largely unexplored opportunity for scaling simulation model development.

3.2. Emerging use of LLMs in simulation and decision support

As part of the broader digital transformation of manufacturing, artificial intelligence has become integral to value creation strategies in industry. Review papers highlight the adoption and potential applications of Gen-AI across the entire value chain from product design to predictive maintenance, process optimization, and quality control, all the way to recycling [27,28]. Zhang et al. [29] present a survey examining the emergence of large-scale foundation models, including LLMs, in manufacturing applications. The study outlines how such models can enhance generalization, improve multi-modal data processing, and support automation across tasks such as process control and simulation modeling. While the potential of LLMs for automating code generation for simulation workflows such as parameter configuration and decision-support is highlighted, still, practical implementations of LLMs in simulation workflows remain limited.

Knowledge-augmented LLMs have recently gained attention as promising tools for automating decision support tasks in industrial contexts. Ma et al. [30] introduced a fault diagnostic pipeline that integrates LLMs with knowledge graphs to enhance fault diagnosis accuracy and reduce hallucination risks. Their study demonstrates that combining structured knowledge representations with LLM-based NLIs can significantly improve the interpretability and reliability of AI-driven reasoning.

Hu et al. [31] proposed a Gen-AI-enabled facility layout design framework that integrates LLMs with knowledge graphs and the Asset Administration Shell (AAS, a standardized digital representation of assets used in Industry 4.0 architectures) to automate layout generation and reduce the complexities of modeling and optimization.

Li et al. [32] test the synergies of LLM-and DT-driven decision making for flexible manufacturing systems. Noticing the issue that LLMs are trained on very generic datasets instead of domain knowledge, the authors built a DT, let the LLM generate decisions encoded in images, which get ingested into the DT for evaluation; that way the LLM can be trained, improve accuracy, and reduce the risk of hallucinations.

Giabbanelli et al. [33] presented a conceptual framework for leveraging LLMs to broaden access to simulations for non-expert users. Their approach outlines a system in which LLMs translate natural language queries into simulation parameters, execute simulations, and generate outputs. While their study emphasizes the potential to scale simulation modeling across domains, applications in manufacturing-specific DES remains largely unexplored.

Dengler et al. [34] introduced a conversational framework for smart energy system simulations, leveraging GPT and their NLP capabilities to translate high-level text descriptions into executable Python scripts that configure simulation scenarios in AnyLogic. While their framework was evaluated within the domain of energy grid modeling, it shares conceptual similarities with the present work. Notably, they identified challenges related to code correctness, realistic data generation, and explainability. These challenges remain critical in extending the approaches to manufacturing simulations.

Obinwanne and Feng [35] conducted a case study demonstrating the feasibility of using LLMs to automate DES modeling. Using two different LLMs and three different prompts, the authors demonstrate the generation of Python code for queuing simulations based on natural language descriptions. The results were compared to implementations built using the traditional simulation language GPSS (General Purpose Simulation System). The study shows that simple DES generations are achievable with the help of prompt engineering and human oversight, while revealing limitations such as inconsistent code and incomplete outputs.

Jackson et al. [4] developed a framework using GPT-3 Codex to automate the generation of simulation models of simple logistics systems into executable Python code given natural language descriptions. The study demonstrated its feasibility and significantly reduced modeling efforts. However, the authors also mentioned issues regarding output inconsistency and model completeness. While their study addressed the ‘prompt-to-code-to-execution’ direction, they focused on simple queuing and inventory control models using text prompts including relevant production context information. The authors noted that the effective prompting with precise language by the simulation engineers becomes critical. The study provided a proof of concept in the logistics domain, and the authors postulated the integration of DTs with LLMs for simulation modeling as a direction for future research, which this work aims to contribute to by using a more holistic simulation workflow with structured data inputs and prompts.

3.3. Identified research gaps and contributions

In summary, prior research has shown that LLMs are capable of translating natural language descriptions into simulation code. While promising demonstrations exist across logistics (e.g., simple queuing simulations) and energy systems, the application of LLMs to automate DES modeling and scenario evaluation in manufacturing remains limited. The majority of studies relied on reviews and conceptual frameworks [27–29,33], few small-scale proofs of concept exists [4,34,35], especially in real manufacturing environments.

Table 1 summarizes how prior work compares with this study. As shown, while several studies demonstrated the feasibility of using LLMs for code generation for simulation scenario creation, none developed an end-to-end practical implementation of a LLM-driven automated generation, adaptation, and evaluation of DES modeling for real-world manufacturing systems. This study extends previous research by developing and validating such LLM-enabled workflow for automated DES modeling with a human-in-the-loop component, thereby addressing a critical gap in the literature. It contributes by:

Table 1
Comparison of related studies utilizing LLMs and the contribution of this work.

Study	Domain	Simulation type	LLM role	Validation	Evaluation of LLM output	Gap addressed by our study
Jackson et al. [4]	Logistics	Queueing and inventory simulations	Text-to-code generation of fictional scenarios	–	Qualitative: executability, correctness of code logic	Did not address DES in manufacturing
Obinwanne and Feng [35]	General simulation modeling	DES	Text-to-code generation of fictional case	–	Quantitative: output correctness vs. GPSS baseline; executability; prompt sensitivity	Limited to simple queueing models, no manufacturing context
Dengler et al. [34]	Energy systems	Scenario generation for AnyLogic	Text-to-code generation of fictional case	–	Qualitative: executability inside AnyLogic, input–output plausibility	Focused on energy systems, not manufacturing simulations
Hu et al. [31]	Manufacturing facility design	–	Knowledge modeling	–	–	Did not address simulation code generation or scenario analysis
Ma et al. [30]	Fault diagnosis in manufacturing	–	Knowledge-enhanced diagnostic reasoning	Validated on industrial datasets	Quantitative: Text similarity metrics of fault detections	Focused on fault diagnosis, not simulation automation
Kmiecik [36]	Logistics	–	Text-to-code generation of genetic algorithm	–	Qualitative: executability, plausibility of produced schedule	No DES modeling or production scenarios
Li et al. [32]	Manufacturing	Digital twin with generative models	Semantic interpretation of human queries	Internal validation based on simulation output	Qualitative: Successful translation of text prompts into target vectors	No automatic simulation code generation or DES modeling
This Study	Manufacturing	DES (productivity and energy efficiency assessments)	Text-to-code generation for real world cases & model-output interpretation	Validated against reference models	Quantitative: code-correctness, accuracy of LLM-generated interpretations and KPIs	Holistic simulation automation of manufacturing systems

GPSS = General Purpose Simulation System.

- Developing a practical implementation of LLMs for manufacturing simulation, spanning model generation, adaptation, and evaluation.
- Testing and discussing practices of Gen-AI techniques for dealing with challenges and opportunities when using LLMs for simulations.

4. Method

This study develops a systematic and automated LLM-driven workflow that, based on a variety of input data, can automatically generate, analyze, adapt, and evaluate DES models of production processes, producing actionable improvement suggestions for production employees. The workflow comprises core steps of simulation-based decision-making for production development [3], enhanced by multiple LLM-driven modules for automatic process execution and a human-in-the-loop mechanism (illustrated in Fig. 1). The LLM-driven workflow was introduced via Python scripts and several LLM calls. The workflow was validated using two case studies from real-world production lines at the case company. A systematic analysis over multiple executions for the two case studies, to test for code correctness and LLM accuracy, is provided.

In the following, the selected LLM agents, LLM workflow techniques, and human-in-the-loop setup are described. Then, the LLM-driven workflow is described step-by-step. The full implementation of the LLM-to-simulation pipeline, including blueprints, LLM calls, prompts, Python scripts, and input data for Case B is available at Github.[1]

4.1. LLM-driven workflow for simulation modeling

4.1.1. LLM agent selection

The LLMs chosen for this work were instances of OpenAI's models GPT-4o, GPT-5-mini, and GPT-5.1, hereafter referred to as *agents*. In principle, the setup is model-agnostic, and other providers' models (e.g., Anthropic or Google) can be integrated in the same way. The authors chose OpenAI because their models were readily available via the case company's enterprise access. OpenAI offers a range of models with differing performance characteristics, which were assigned to programming and interpretation tasks within the workflow according to their respective strengths; their benchmark results are shown in Table 2. At the time of the experiments, GPT-5.1 was the best-performing model among the evaluated OpenAI models on programming-oriented benchmark tasks. It handled the most demanding tasks of building the initial simulation model (builder- and inspector-agents). Its predecessor, the GPT-5-mini model was a reduced, more cost-efficient version. It was used for simpler tasks and where computational efficiency mattered most, such as for converting the initial simulation model into flow charts (visualizer-agent), for creating parametric or structural changes (adaptor- and inspector-agents), and for natural language summaries (evaluator- and CPD-agents). Finally, the GPT-4o model was chosen due to its versatility in generating structured output, used for generating JavaScript Object Notation (JSON) instructions from the BT analysis (bottleneck-agent).

Table 2
The resolved rates from the Software Engineering Benchmark (SWE-bench), designed to test LLMs on their software engineering capabilities, and API costs across OpenAI's models [37–40]. GPT-5.1 was used for the most complex tasks, e.g., model generation; GPT-5-mini was used for simpler tasks, e.g., model adaptations; GPT-4o was used for tasks that benefited from structured output, e.g., generating instructions. Data on additional models is provided as reference for quality and costs.

Model	% Resolved	Costs (USD/1M tokens)			Structured output
		Input	Cached Input	Output	
GPT 5.1	**76.30**	$1.25	$0.13	$10.00	No
GPT-5	65.00	$1.25	$0.13	$10.00	No
GPT-5-mini	59.80	**$0.25**	**$0.03**	**$2.00**	No
o3	58.40	$2.00	$0.50	$8.00	No
GPT-4.1	39.58	$2.00	$0.50	$8.00	No
GPT-4o	21.62	$2.50	$1.25	$10.00	**Yes**

4.1.2. LLM workflow techniques

Different Gen-AI techniques were applied in the workflow to increase computational efficiency, cost efficiency, and output consistency. These include zero-, one-, and few-shot implementations, prompt engineering, evaluator–optimizer workflows, and human-in-the-loop, chosen based on best practice and qualitative observations of robustness for solving the task [13]. Zero-, one-, and few-shot implementations refer to the amount of context given to the LLM, from which it learns. The builder-agent is guided by a blueprint model that serves as a single example of the desired structure, resulting in a one-shot implementation. In contrast, the adaptor-agent performs adaptations based only on the initial model and a natural-language instruction, without any example of how such adaptations should be carried out, and therefore operates in a zero-shot manner. Prompt engineering is a

[1] https://github.com/thomasschmitt1993/LLM-DES-Modeling

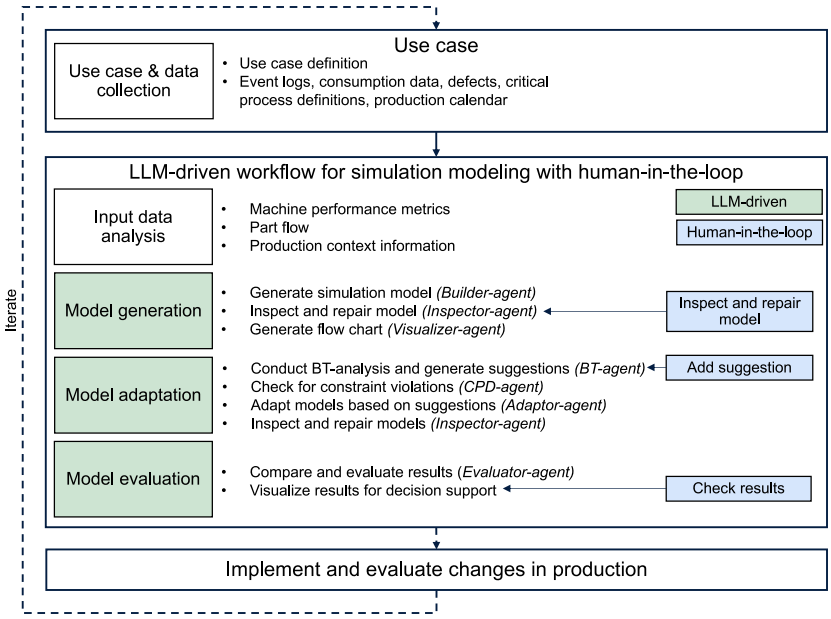

Fig. 1. Overview of this study's LLM-driven workflow with a human-in-the-loop mechanism to automate the generation, adaptation, and evaluation of simulation models. The framework was tested on two real-world production lines. Implementing the suggested changes in production fell outside the scope of this study. *Source:* Adapted from Schmitt et al. [3].

technique that iteratively improves prompts to narrow down the task. It is applied so to limit the risk of hallucinations and increase output reliability. The evaluator–optimizer technique is used where one LLM call generates and another inspects and, if necessary, repairs the code.

4.1.3. Human-in-the-loop

Lastly, the interaction between human experts and the agentic system is a critical component of the proposed framework. The framework design explicitly integrates a robust human-in-the-loop component for ongoing supervision, feedback, and adjustments. The clear allocation of roles ensures that human experts oversee strategic and high-level decision-making, while LLMs handle routine, repetitive, and technically complex simulation tasks. In this study, automation refers to the execution of the simulation modeling workflow, while human-in-the-loop checkpoints serve as governance mechanisms to ensure reliability rather than as functional dependencies of the method. Fig. 1 illustrates the human-in-the-loop approach, described in the following:

1. **Mandatory inspection of the initial simulation model.** The human operator inspects if the initial model generated based on the input data is correct, and repairs it if necessary. This is important, as the model generation is the most complex and comprehensive task that the agent needs to perform, and therefore the most error-prone. In addition, all following processes, including the execution time and costs, are determined by the initial model.
2. **Optional improvement suggestion.** The human operator can add a suggestion in addition to those made by the agent. This is to enhance the exploration, as human operators possess a broader understanding and further domain knowledge, which the LLM does not have access to. This can lead to better suggestions in solving the BT.
3. **Mandatory check of results.** The human operator verifies the interpretations made by the agent from comparing the models' results. This is to oversee potential hallucinations by the LLM.

4.2. Use case and data collection

Simulation-based production development begins with clearly defining the production scenario and collecting relevant data. Two real-world production lines from a large automotive manufacturer in Sweden served as the test cases. To evaluate the performance of the lines in terms of productivity, energy efficiency, and inventory, the following metrics were defined: Throughput (TH) — total number of parts produced divided by production time [parts/h]; (ii) Work-in-process (WIP) — mean number of parts in the system's buffers and machines; and (iii) Specific Energy Consumption (SEC) — total energy consumed divided by the number of produced parts [kWh/part]. To build an accurate simulation model for capacity- and energy efficiency assessments, the following information is required: machine performance metrics in terms of productivity and energy consumption, part-flow logic (e.g., serial or parallel operations), buffer capacities, defect rate, production shift calendar, and knowledge on the limitations of process improvements (e.g., constraints such as minimum processing time of certain operations).

The collected data included event logs from the manufacturing execution system (MES) to extract cycle times (CTs), availability metrics, mean-time-to-repair (MTTR), and the part-flow logic. For Case A, the event logs were pre-processed by excluding erroneous events (e.g., overnight stops). The part-flow logic for Case A was derived from a file documenting an example flow of a unique part through the system. Case B, as described in detail in Schmitt et al. [3], represents a more complex production line; for this case, a simplified mock-up event log with corresponding part-flow logic was generated. This was done to lower model complexity and allowed sharing the Case B-related data alongside the code via GitHub[1]. For Case A, power consumption during idling/stopped and processing was approximated from supplier specification documents, while for Case B it was manually measured. Information on the number, sequence, and capacity of machines and buffers were obtained through production visits and documents. Defect rates were extracted from a separate system, and fictitious critical process definitions (CPDs), stating requirements on process parameters, were added, as original documents were not available. These CPD-statements were generated with the help of a simulation expert at the company. An excerpt of an event log is shown in Table 3.

Table 3
Example event log from Case B with manually added average power ratings per machine state ($\bar{P}$).
Source: Adapted from Schmitt et al. [3].

Equipment	Machine state	Start time	End time	$\bar{P}$ [kW]
Press 1	Idle	13:44:15.78	13:44:28.83	1.25
Press 1	Working	13:44:28.83	13:45:28.42	1.28
Handling	Idle	13:44:30.09	13:45:53.34	0.50
Handling	Working	13:45:53.34	13:47:12.95	0.74
Washing	Working	13:46:10.81	14:20:05.20	35.34

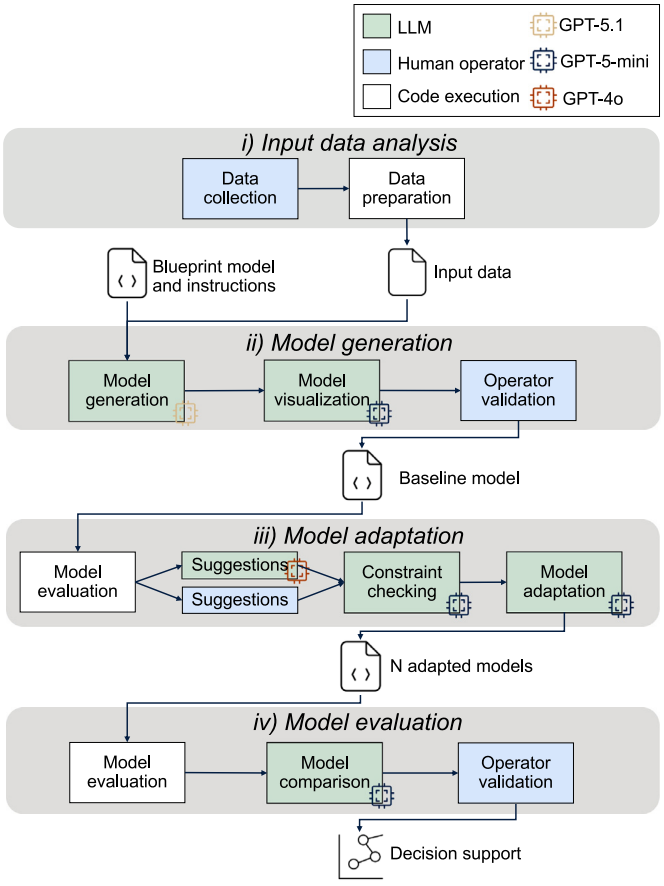

Fig. 2. Workflow of the proposed LLM-assisted simulation modeling approach. The process comprises model generation, model adaptation, and model evaluation, combining automated LLM-based steps with human validation to support simulation-based decision-making. All LLMs were executed using their default API parameters, including standard context limits and internal sampling settings for, e.g., temperature and top_p.

4.3. LLM-workflow stages

The complete AI workflow utilizes the described LLMs and LLM techniques. It consists of four main steps: (i) input data analysis, (ii) model generation, (iii) model adaptation, and (iv) model evaluation, as shown in Fig. 2. The first step, input data analysis, was done using static Python code, while the three modeling-related steps were LLM-driven.

4.3.1. Input data analysis

This step prepares all necessary information into machine-readable format. All machine performance metrics (CT, availability, MTTR, power ratings during idling and processing) are calculated using Python scripts. The part-flow logic was extracted from the part movement logs using the Python-based process mining library PM4Py [41].

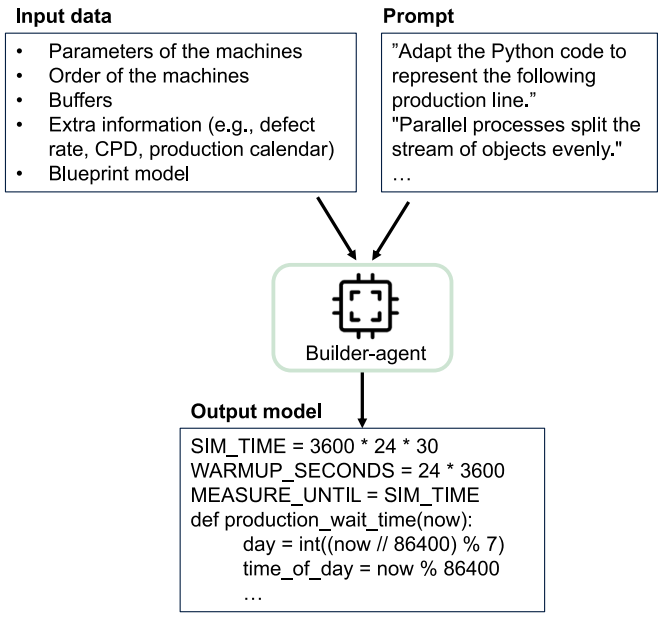

Fig. 3. Input data and instructions given to the builder-agent in order to generate a current-state simulation model.
Source: Adapted from Gegenmantel [43].

To accommodate limitations in data availability for automatic parameter extraction (e.g., details regarding buffer capacities, variants, or specific custom logic), the module also accepts manual inputs. Thus, the remaining information on defects, production shift times, and CPDs were compiled into text.

4.3.2. Model generation

A *builder-agent* translates the input data to generate the current-state simulation model using SimPy, a Python-based DES library [42]. The generation process is guided by a predefined, generalized blueprint template, together with a structured set of parameters defining the production configuration. Using natural-language instructions, the builder-agent adapts the blueprint to the provided input and produces the corresponding Python code. An *inspector-agent* then verifies the generated code, repairing errors when detected. After this automated check, the user is prompted to manually review and adjust the model, allowing additional refinement of the current-state representation. The output of this stage is an executable Python simulation model containing operations, buffers, machine states, and initial parameterization. Fig. 3 illustrates the data input and code-generation process performed by the builder-agent. The resulting model is subsequently passed to a *visualizer-agent*, which converts the Python code into JavaScript syntax to produce a process-flow diagram, supporting visual inspection and error identification. Lastly, the simulation model is executed for a number of independent replications, defined via replication analysis, using distinct random seeds. A warm-up period was applied to mitigate initialization bias; the warm-up duration was defined based on a steady-state analysis. Results of key metrics (TH, SEC, and WIP) for each case were collected over a 30-days and 7-days simulation horizon, respectively.

4.3.3. Model adaptation

Following the model generation, a predefined BT-analysis is performed on the current-state simulation model using utilization analysis. BTs are machines where an infinitesimal improvement in one machine has the largest effect on overall system performance and thus are the target for process improvements [44]. The ranked BTs are provided to

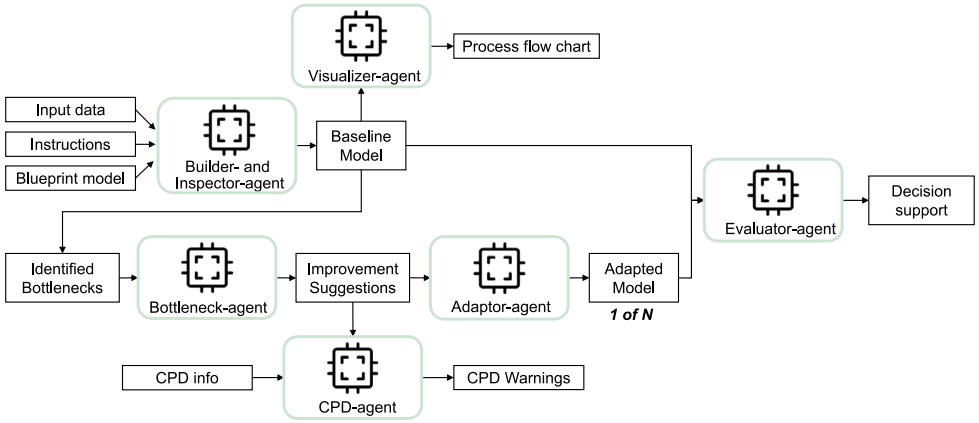

Fig. 4. Overview of the workflow. A total of 6 + N API calls are made, where N is the number of improvement suggestions. *Source:* Adapted from Gegenmantel [43].

a *bottleneck-agent*, which generates a set of improvement suggestions. Note that the utilization-based bottleneck detection is included solely to provide structured input for the LLM; the study does not aim to compare or advance BT detection methods. A human operator can manually evaluate the suggestions and specify additional improvement suggestions via text. This is crucial, as the LLMs do not have access to context-specific constraints such as layout, available space, or cost considerations, which are therefore assessed by the human operator. In contrast, explicit process constraints such as maximum buffer capacities are provided to a *CPD-agent*, which checks whether the suggested changes violate CPDs and passes information to the human operator via text. Next, each suggestion is separately implemented by an instance of the *adaptor-agent*, outputting a new adjusted simulation model per suggestion. By default, three adapted simulation models result, however, the number of suggestions is configurable. Each model code is checked using the *inspector-agent* and, if necessary, repaired. Adaptations fall into one of three 'complexity' levels; (i) low complexity for only parameter-level adaptations (such as adjustments in buffer capacities and availabilities), (ii) intermediate complexity for structural adaptations (e.g., adding or removing operations), and (iii) high complexity for modifying the underlying operational logic (e.g., implementing batch processing) [45].

4.3.4. Model evaluation

Following each model adaptation, the simulation model is run and key metrics (TH, WIP, and SEC) are averaged across replications and visualized in a bar chart for at-a-glance comparison between the initial and the adapted models. Then, an *evaluator-agent* assesses the effectiveness of adaptations by comparing the results of the generated scenarios to the current-state and to each adapted simulation model. The evaluations are structured into textual summaries to present human decision-makers with actionable recommendations, including balanced trade-offs among potentially conflicting objectives such as energy efficiency improvements and inventory levels. The decision-maker then verifies the results and evaluates these trade-offs. An overview of the workflow is provided in Fig. 4.

4.4. Workflow validation

Validation of the LLM-driven workflow was done via frequent discussions with a simulation expert at the company, by applying the workflow to two actual production scenarios, and through systematic analyses across multiple workflow executions for the production scenarios. Two production lines, an automated assembly line and an automated machining line, were used to evaluate the reliability, consistency, and accuracy of the LLM integrations. For each case, challenges such as code errors, type of errors, hallucinations, and the need for human oversight were documented and addressed using the Gen-AI techniques described in Section 4.1.

To assess the validity and reliability of the LLM-driven workflow and its automatically generated simulation models, we followed established verification and validation principles, including those recommended by Sargent [46]. Verification ensured that the generated model logic, flows, and assumptions were consistent with the intended reference model. Validation included the comparison of the results of the generated models against equivalent models built in FACTS Analyzer simulation software. For this comparison, the Python-based models resulting of the first experiment workflow of each case were re-built using the simulation software, and the resulting outputs were compared. The choice of simulation software was based on previous experience and the ability to incorporate energy consumption data, although any tool with comparable functionality (e.g., Siemens Plant Simulation) could be used. Input–output validation was performed by comparing model outputs using means and 95% confidence intervals based on multiple independent simulation runs. As suggested by Sargent [46], acceptable accuracy ranges for KPIs (TH, SEC, and WIP) were defined, and the same input data were used across all models to maintain consistency. Minor deviations between Python-based and software-based results were anticipated due to differences in how simulation tools initialize operations, and model validity was confirmed when discrepancies remained within a defined acceptable threshold (5%–10%).

5. Results

To evaluate the capabilities of the designed Gen-AI framework, two case studies on different production lines at a large Swedish automotive manufacturer were performed. The experiments focused on validating the ability of the LLM-driven workflow to automate the creation, adaptation, and evaluation of DES in manufacturing. The two case studies differ in amount and organization of operations, where Case A consists of four operations organized in series and Case B consists of seven operations, ordered in both linear and parallel fashion. Both cases were tested over ten and eleven experiment runs, respectively. After the experiments of Case A, the workflow was expanded with an additional agent (for generating process flow charts) and several minor adjustments (e.g., prompting). In the following, at first the blueprint model is introduced, then the LLM API call is illustrated, after which both Case A and Case B results are shown and compared. Lastly, a validation step that compares the Python-based models with FACTS Analyzer models is conducted for each case.

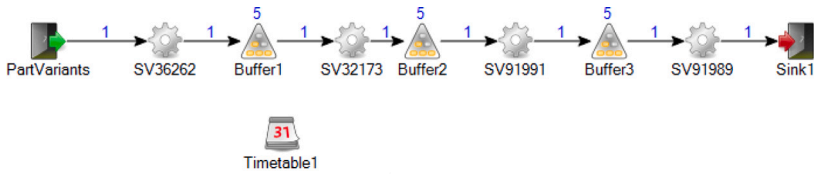

Fig. 5. Case A production line modeled in FACTS Analyzer. Simulation parameters such as simulation horizon (30 days), warm-up period (1 day), and replications (25, relative error of 2%) were retrieved through steady-state and replication analysis.

Table 4
Machine performance and power ratings for Case A. Buffers between consecutive machines are identical (capacity 5 parts; delay 10 s).

Machine	CT [s]	Availability [%]	MTTR [s]	Idle power [kW]	Proc. power [kW]
SV36262	320	85	1409	10	50
SV32173	290	85	718	10	50
SV91991	134	98	6717	10	50
SV91989	41	99	671	10	50

5.1. The generic blueprint model

The blueprint model, perhaps this study's most crucial element, provided the expected structure of a production model and thus supported a one-shot adaptation by the builder-agent. The builder-agent, fed with all relevant production data, fetched the blueprint model and conducted adaptations in order to fit it to the real-world production case. The blueprint model consisted of numerous functions, instructing among others the implementation of a production schedule; the setting of merger and splitter processes before and after parallel stations; the initialization of part generation and part movement; the creation of buffer and machine instances and their respective classes; the underlying exponential distribution used for failures and MTTR; and the calculation of KPIs. An excerpt of the blueprint model is shown in Listing 1 in Appendix A, the full code is available at Github[1].

5.2. LLM API calls

During the execution of the LLM workflow, a total of 6 + N API calls are made, where N is the number of improvement suggestions (Case A utilizes 5 + N calls, as Case B uses an additional visualizer-agent for visualizing and verifying the process flow). These agents are fed with context information (e.g., the blueprint code and information on production flow and machine performance metrics) and instructions (prompts). Two examples are provided: in Listing 2 in Appendix B, the call and prompt made for the builder–agent are shown; Listing 3 in Appendix B shows the function for model adaptations: The current-state simulation model (original_code) and the instruction (prompt) are provided to the adaptor-agent.

5.3. Case A - An automated machining line

The production line in Case A comprises four sequential operations with identical intermediate buffers between consecutive machines. Due to its straightforward layout, this line was selected as the initial case study. The model is illustrated in Fig. 5. In total, ten experiments were conducted on Case A. In the following subsection, the results for the first experiment are shown in greater detail, while the overview of the results across experiments is shown in Section 5.3.4.

5.3.1. Input data for Case A

Performance metrics for each machine were extracted using the procedure described in Section 4.3.1. Information that could not be extracted from event logs (buffer capacities and delays, machine sequence, scheduled production breaks from Friday 17:00–Saturday 07:00 and Saturday 17:00–Sunday 07:00, and power ratings from supplier specification sheets) was added manually. The same two CPD statements were added for both cases, of which only the second affects Case A: (1) the presses need to have a process time of at least 60 s, and (2) all buffer capacities must be kept at the same original level. The machine sequence and performance parameters are summarized in Table 4.

Terminal Output

```
----Results from model: Original model

=== Mean Overall KPIs over 25 runs ===
Throughput = 7.38 parts/hour, WIP = 4.47 parts, Mean Energy
    Consumption per Part = 14.2158 kWh/part
```

Fig. 6. Terminal output showing the results of the current-state model for Case A. All KPIs are reported as mean values over 25 replications.

5.3.2. Model generation for Case A

Next, the builder-agent, using the provided production data, instructions, and blueprint model, generated the current-state simulation model. Afterwards, the inspector-agent verified code correctness. Before execution, the workflow prompts the human operator to review and, if necessary, refine the generated model. A manual fix was necessary for one experiment (Experiment #7). Listing 4 in Appendix C shows an excerpt of the resulting simulation model, including the initialization of machines and parameters. The model was then executed, and KPIs were obtained; the results are shown in Fig. 6.

5.3.3. Parametric adaptations for Case A

The utilization analysis embedded in the blueprint consistently identified the following three BTs: SV36262, SV32173, and SV91991. Based on these findings, the bottleneck-agent proposed three improvement strategies, shown in Fig. 7.

These suggestions were compared to CPD information by the CPD-agent. The CPD-agent flagged a violation in the first adaptation: increasing buffer capacities conflicts with the requirement to maintain original buffer sizes. The relevant output is shown in Fig. 8.

Each suggestion was implemented in a separate model by the adaptor-agent, yielding three adapted versions. All adapted models were saved and executed to obtain KPIs, shown in Fig. 9.

The evaluator-agent interpreted the results and selected adapted model version 2 as the most effective configuration, achieving the highest TH and lowest SEC with only a modest increase in WIP. Should WIP-considerations be important, the agent suggests version 3 with the

Terminal Output

```
{'instructions': ['Increase the capacity of buffer1, buffer2, and
    buffer3 to 10.', 'Increase the availability of SV36262 to
    90.0.', 'Increase the availability of SV32173 to 90.0.']}
Do you want to manually add one change to the model? If yes please
    answer with the change (leave blank to skip):
```

Fig. 7. Bottleneck-agent output suggesting three improvements. A fourth suggestion can be added by the user, but no user input was provided in this example.

Terminal Output

```
Safety Inspector activated:
1st: "Increase the capacity of buffer1, buffer2, and buffer3 to 10."

This change conflicts with critical process 2 (all buffer
    capacities must be kept at the same original level). The other
    two changes do not match any listed critical process.
```

Fig. 8. CPD-agent output flagging process requirement violations for the first improvement suggestion. The second and third suggestions were correctly identified as non-critical.

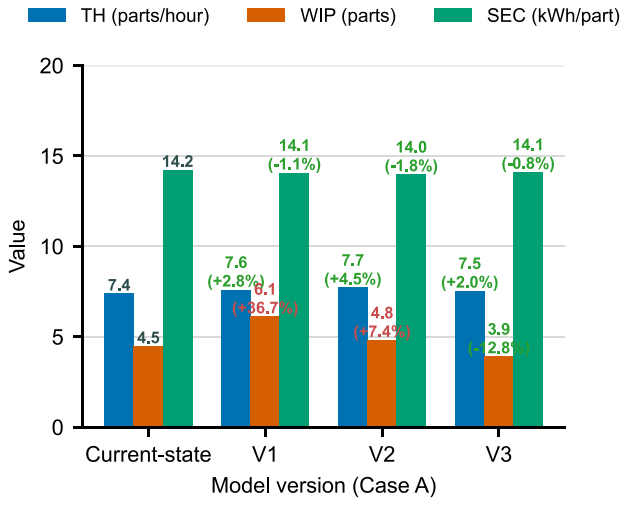

Fig. 9. Summary and comparison of model performances for Case A in terms of TH, WIP, and SEC.

lowest WIP and modest improvements in TH and SEC. Version 1 and 4 offered only smaller improvements in TH and SEC while raising WIP substantially. The evaluator-agent's output is shown in Fig. 10.

5.3.4. Experiment results for Case A

The results of the LLM-driven simulation modeling for Case A across ten experiments are shown in Table 5. Eight runs were first-pass. Experiment #6 failed due to a logic error, caused by the way the LLM had defined the production hours of the model, crashing as soon as the second week started due to a negative delay value. Experiment #7 resulted in an invented parallel operation.

In the next stage, the bottleneck-agent created relevant suggestions across experiments, focusing on buffer capacities around the BTs and the availability, CT, and MTTR of the BTs. One BT suggestion in experiment #7 suggested a doubling of the source capacity from 1000 to 2000, not resulting in any KPI change. All other adaptations showed modest to larger improvements in KPIs. The adaptations executed by the adaptor-agent were successful in all instances but one (Experiment

Terminal Output

```
Interpretation and recommendation
- Version 2 delivers the best combined improvement: highest
    throughput and best energy-efficiency, with only a small
    increase in WIP relative to the original. That makes it the
    preferred choice for throughput- and energy-focused objectives.
- Version 3 is the choice if lowering WIP is critical (inventory,
    lead-time or floor-space concerns).
- Version 1 increases throughput slightly and reduces energy a
    little, but raises WIP substantially, so it is less attractive
    unless there is a specific reason for that configuration.

Suggested next steps
- If differences matter operationally, perform statistical testing
    or increase the number of runs to confirm significance (the
    changes are modest).
- If you have objective weights (e.g., throughput vs WIP vs
    energy), compute a weighted score to pick the best
    configuration automatically.
```

Fig. 10. Evaluator-agent output identifying adapted model version 2 as the preferred improvement in Case A, including suggestions for next steps.

#2), where the second adaptation run exceeded the manually set timeout (> 300 s). Across all experiments, the CPD-agent correctly flagged the process violations in each experiment, and the evaluator-agent correctly interpreted the results and weighed off between the models' results.

5.4. Case B - An automated assembly line

The production line in Case B comprises seven sequential machines, with the exception of two presses operating in parallel. This line is a simplified representation of the production system studied in detail by Schmitt et al. [3]. Identical buffers are located between each consecutive operation. The modeled production line is illustrated in Fig. 11. The workflow applied in Case B followed the same procedure as in Case A, with few enhancements: (1) the case considers defect rates and defect sinks; (2) the authors added a suggestion for structural modification to the bottleneck-agent; (3) a visualizer-agent translates the python model into a process flow chart; (4) prompts were refined for the CPD-agent. In the following, the results for Experiment #7 are shown in greater detail, while readers are referred to Section 5.4.4 for an overview of the results across experiment runs.

5.4.1. Input data for Case B

Information on station configuration and performance metrics was compiled for the builder-agent in the same way as for Case A. Additionally, in Case B, a defect rate of 8.9% at the Quality station was applied. The same CPD information as for Case A was provided in Case B. The sequence and performance parameters for Case B are shown in Table 6.

5.4.2. Model generation for Case B

Based on the inputs, the builder-agent generated the model, the inspector-agent again verified code correctness. An excerpt of the resulting simulation model for Experiment #7 is shown in Listing 5 in Appendix D. The visualizer-agent translated the simulation model into JavaScript to create process flow visualizations. The resulting part flow is shown in Fig. 12. The code was then executed and the KPIs of the current-state model are shown in Fig. 13.

5.4.3. Parametric and structural adaptations for Case B

The utilization analysis identified the same three BTs across experiments: Press 2, Press 1, and the Quality station. Corresponding BT improvement suggestions were automatically generated and im-

Table 5
Results for code correctness and LLM accuracy for Case A across 10 experiments. Text in red indicates LLM errors. Each successful experiment resulted in four executable simulation models, requiring eight API requests (avg. ~24,500 tokens; cost $0.13). The average execution time of the last three successful experiments was 1190 s (≈20 min).

Run	Model generation					Model adaptation			Model evaluation
	Model result	Error type	KPIs			BT suggestions	CPDs flagged	Adaptation result	Evaluation result
			TH	WIP	SEC				
1	✓	–	7.38	4.47	14.22	1. Buffers[a] BT1–2 2. AVB BT1 3. AVB BT2	✓	✓	✓
2	✓	–	7.40	4.52	14.21	1. Buffer BT1 2. CT BT2 3. MTTR BT1	✓	Timeout	–
3	✓	–	7.38	4.47	14.22	1. Buffer BT1 2. CT BT1 3. AVB BT2	✓	✓	✓
4	✓	–	7.40	4.52	14.21	1. CT BT1 2. Buffer BT1 3. AVB BT2	✓	✓	✓
5	✓	–	7.38	4.47	14.22	1. AVB BT1 2. Buffer BT1 3. MTTR BT2	✓	✓	✓
6	Failed	Scheduling	–	–	–	–	–	–	–
7	Manual correction	Structural	7.38	4.47	14.22	1. Buffer BT1 2. AVB BT1 3. Source capacity	✓	✓	✓
8	✓	–	7.38	4.47	14.22	1. Buffers BT1–2 2. CT BT1 3. AVB BT2	✓	✓	✓
9	✓	–	7.38	4.47	14.22	1. Buffer BT1 2. AVB BT1 3. CT BT2	✓	✓	✓
10	✓	–	7.38	4.47	14.22	1. Buffers BT1–2 2. CT BT1 3. MTTR BT1–2	✓	✓	✓

[a] All 'buffer' references indicate input buffer capacities.

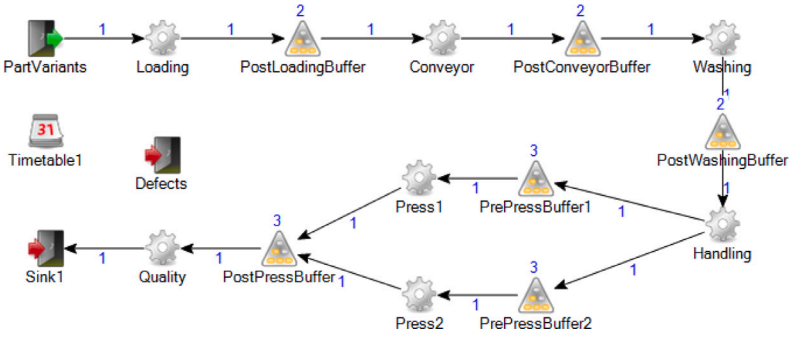

Fig. 11. Case B production line modeled in FACTS Analyzer. Simulation parameters such as simulation horizon (7 days), warm-up period (1 day), and replications (10, relative error of 1%) were retrieved through steady-state and replication analysis.

Table 6
Machine performance and power ratings for Case B. Buffers between stations are similar: PostLoadingBuffer, PostConveyorBuffer, and PostWashingBuffer with capacity of 2 parts and delay of 10 s; PrePressBuffer 1&2 and PostPressBuffer with capacity of 3 parts and delay of 32 s.

Machine	CT [s]	Availability [%]	MTTR [s]	Idle Power [kW]	Proc. Power [kW]
Loading	12	90.5	68	0.25	0.72
Conveyor	6	100	60	0.00	0.00
Washing	14	80.9	269	4.28	35.24
Handling	25	97.8	74	0.50	0.74
Press1	175	87.8	73	1.25	1.28
Press2	176	87.7	74	1.25	1.27
Quality	41	85.9	66	0.58	0.84

plemented, each addressing parameters of BT machines or adjacent buffers. These suggestions varied slightly from experiment to experiment. To further challenge the adaptation process and evaluate the flexibility of the adaptor-agent, one additional manually defined suggestion was introduced to each experiment. In total, four suggestions were implemented, shown in Fig. 14.

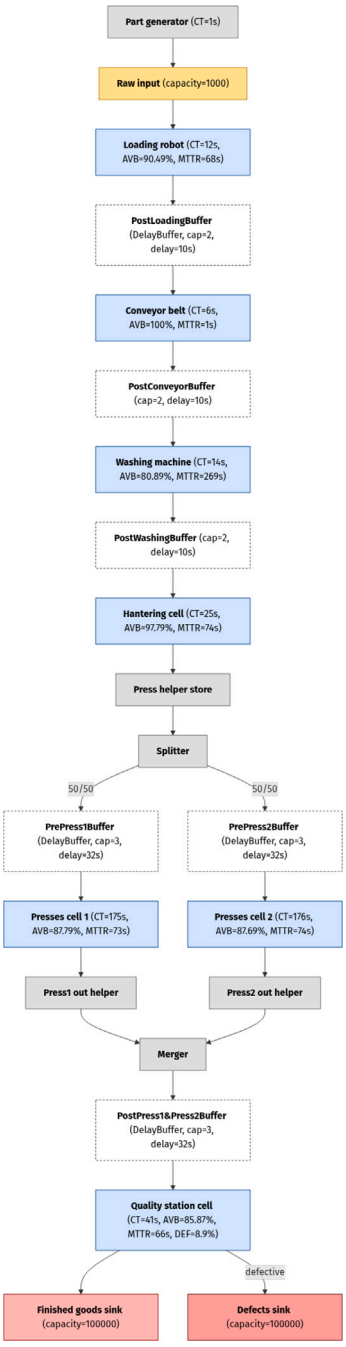

Fig. 12. Flow chart of Case B created by the visualizer-agent using JavaScript-based Mermaid syntax and rendered into a diagram using a helper function.

Terminal Output

```
----Results from model: Original model

=== Mean Overall KPIs over 10 runs ===
'Throughput = 27.26 parts/hour', 'WIP = 18.22 parts', 'Mean Energy
    Consumption per Part = 0.4510 kWh/part'
```

Fig. 13. Terminal output showing the results of the current-state model for Case B. All KPIs are reported as mean values over 10 replications.

Terminal Output

```
{'instructions': ['Increase the capacity of pre_press1_buffer and
    pre_press2_buffer to 5.', 'Increase the process_time of Presses
    cell 1 and Presses cell 2 to 160.', 'Reduce the defect_rate of
    quality_station to 0.07.']}
Do you want to manually add one change to the model? If yes please
    answer with the change (leave blank to skip): Add a third
    parallel press with its own pre-buffer instance, with the same
    capacity and delay as the existing pre-buffers. Reuse the
    existing post-press buffer.
Added human input.
```

Fig. 14. Bottleneck-agent output suggesting three improvements by default, with an optional fourth suggestion added manually by the user. In the second suggestion, the agent incorrectly stated to *increase* instead of *decrease* process time, however, its suggestion to change the process time value to 160 s constitutes a decrease. In the next step, the adaptor-agent correctly implemented the change towards a lower value of 160 s.

Terminal Output

```
Safety Inspector activated:
1st - "Increase the capacity of pre_press1_buffer and
    pre_press2_buffer to 5."
- Why critical: This directly changes buffer capacities, which
    violates critical process #2 ("All buffer capacities musst be
    kept at the same original level"). This is a critical change
    and should be rejected or reviewed.

2nd - "Increase the process_time of Presses cell 1 and Presses cell
    2 to 160."
- Why critical: This affects the presses' process_time, which is a
    monitored critical parameter (critical process #1 requires
    processtime >= 60s). This change is part of a critical process;
    note that it does not violate the rule because 160 >= 60 (so it
    is acceptable from that constraint).

4th - "add a third parallel press with its own pre-buffer instance,
    with the same capacity and delay as the existing pre-buffers.
    reuse the existing post-press buffer."
- Why critical: This adds a pre-buffer instance (affects the
    buffering stage). Although the new pre-buffer is stated to have
    the same capacity (so it does not violate the "keep capacities
    at original level" constraint for existing buffers), it still
    touches the buffer/press subsystem and is therefore part of the
    critical processes and should be reviewed for side effects on
    flow, synchronization and system behavior.

3rd - "Reduce the defect_rate of quality_station to 0.07." - not
    part of the listed critical processes.
```

Fig. 15. CPD-agent output flagging process requirement violations for the first improvement suggestion, mentioning the non-violated process time requirement in connection with suggestion 2, and highlighting the need to critically assess the effect of the fourth suggestion on critical constraints. The second and third suggestions were correctly identified as non-critical.

To ensure compliance with critical process definitions, the CPD-agent evaluated the proposed adaptations, correctly highlighting the violations of the first adaptation while suggesting to critically assess the fourth suggestion (see Fig. 15). The results and KPI comparison between the current-state and adapted models are shown in Fig. 16.

The evaluator-agent interpreted the results and identified adapted model version 4 as the most effective configuration. It provided the highest TH and lowest SEC, however, it also resulted in the highest WIP. The evaluator-agent further stated that, if keeping WIP at current level was important, version 2 would be the best compromise with modest TH and SEC improvements. It noted that versions 1 and 3 show only minor improvements, while version 1 significantly increased WIP; these models are therefore not attractive. In addition, the agent provided suggestions on next steps, e.g., validating the choice with additional runs for more statistically robust results. The output of the evaluator-agent is shown in Fig. 17.

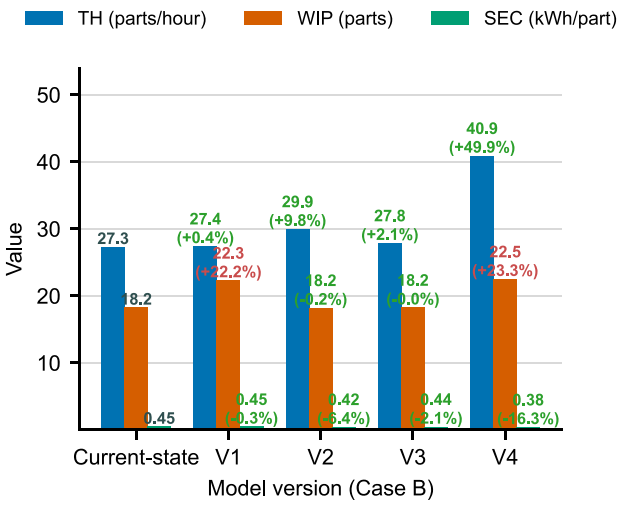

Fig. 16. Summary and comparison of model performances in terms of TH, WIP, and SEC.

Terminal Output

```
Which configuration is best?
- If your primary objective is maximum throughput (and lower energy
    per part), Adapted version 4 is clearly the best: it increases
    throughput by ∼50% and reduces energy per part by ∼16%. The
    trade-off is a higher WIP (+23%).
- If you must keep WIP at or near the baseline level while
    improving throughput and energy, Adapted version 2 is the best
    compromise: ∼9.8% higher throughput, virtually unchanged WIP,
    and ∼6.3% lower energy/part.
- Versions 1 and 3 show only minor improvements in throughput and
    energy; v1 increases WIP substantially and so is not attractive
    unless that WIP increase is acceptable.

Recommendation and next steps
- Choose Adapted version 4 if throughput and energy reduction are
    top priorities and the higher WIP is acceptable.
- Choose Adapted version 2 if you need to hold WIP near current
    levels while improving throughput and energy efficiency.
- Validate whichever choice with more runs and statistical testing
    (confidence intervals, hypothesis tests) and, if needed,
    explore ways to lower WIP in version 4 (buffer sizing, pacing
    adjustments) or further tune version 2 for larger throughput
    gains.
```

Fig. 17. Evaluator-agent output suggesting adapted model version 4 as favorable, if TH and SEC are priorities while increased WIP can be accepted. Otherwise, if WIP shall be held at current levels, then adapted model version 2 can be chosen with improved TH and SEC.

5.4.4. Experiment results for Case B

The results over eleven experiments for Case B are shown in Table 7. Seven runs were first-pass; four runs necessitated manual corrections. One model (Experiment #3) required a minor parametric adjustment of a helper buffer capacity, which caused deviations in KPIs. Two models (#4 and #10) required more elaborate repairs of structural issues about the way the LLM modeled the splitting of parts between the handling station and the two presses using helper buffers and a splitter logic. In both cases, routing errors were introduced which looped parts from the helper store into extra queues and back into the helper store, before forwarding them to the presses.Lastly, Experiment #11 resulted in two errors: the first error concerned the confusion of helper stores and actual buffers, which resulted in wrong WIP values, and the second concerned an unintended production break, where Saturday's break hours were also applied to Friday. The subsequent workflow executed correctly across all runs: The LLM correctly identified BTs and suggested relevant improvement, CPD violations were flagged, and all four improvement suggestions were correctly implemented. The interpretation of the results was also correct, and trade-offs across adapted models were highlighted.

5.5. Summary of experiment results

Case studies A and B showed that the LLM-driven workflow can automatically generate discrete-event simulation models, propose and implement modifications, report CPD violations, and evaluate multiple adaptations. Still, errors occurred, with the model generation phase being the most error-prone: Across 21 experiments, one run crashed and five required manual corrections to modeling errors. Case B exhibited more structural errors, particularly in splitting and merging part flows at the parallel station. Model adaptation was more reliable: most adaptations were implemented correctly. Two exceptions occurred: in Case A, one experiment timed out during the second adaptation step; in Case B, one BT suggestion contained a logical inconsistency. Across experiments, the workflow consistently detected CPD violations and suggested relevant modifications, such as adjustments to buffer capacities, machine availability, and processing times at BTs. The evaluator-agent produced structured performance analyses and coherent summaries. Fig. 18 presents an overview of code correctness and interpretation accuracy for both cases.

5.6. Validating the worfklow with simulation software

To assess the validity of the LLM-generated Python-based models for Case studies A and B, the first experiment of each case was compared against manually built models in the commercial simulation software FACTS Analyzer, using TH, SEC, and WIP as metrics. Tables 8 and 9 summarize the relative deviations between the Python-based and the C++-based models. For Case A, TH deviations were approximately −4% and SEC deviations around 2%, both with little spread across models. WIP deviations averaged roughly 4%, with a wider spread ranging from about 1% to 7% across models. These discrepancies likely stem from the structural characteristics of Case A: the line consists of only four operations, with the first station acting as a clear CT-determined BT. As a result, WIP rarely accumulates unless the third station (highest MTTR) experiences a long failure, making WIP highly sensitive to stochasticity and to differences in how FACTS Analyzer and SimPy model failure and repair distributions. Such sensitivity can amplify small stochastic or implementation-level differences between tools. In Case B, deviations were modest. Across metrics, average discrepancies remained below 1%: approximately 0.8% for TH, 0.3% for SEC, and −0.4% for WIP.

Consistent with our predefined validation criteria, the model was considered sufficiently validated when (i) the observed reference value fell within the simulation's confidence interval or deviated by less than 10%, and (ii) no systematic bias was detected. Taken together, the deviations observed across both case studies fall within commonly accepted thresholds in simulation practice and align with guidelines by Sargent [46] for acceptable model accuracy. These results support the credibility and practical validity of the LLM-driven workflow.

6. Discussion

Simulation modeling is widely applied in industry to analyze and improve manufacturing systems. Generative AI offers new opportunities to automate the generation, adaptation, and analysis of simulation models by leveraging NLP and code-generation capabilities, potentially lowering the expertise barrier for non-specialists or domain experts [26]. Earlier studies have indicated the potential of LLM-assisted model generation [4,35] and LLM-enhanced DTs [32], but so far no studies have demonstrated and validated a complete end-to-end workflow spanning model generation, adaptation, and evaluation. This study addresses this gap by presenting such a workflow and evaluating its application in two real-world manufacturing cases.

Table 7
Results for code correctness and LLM output accuracy for Case B across 11 experiments with a 7-day simulation horizon, 1-day warm-up, and 10 replications. Text in red indicates errors in the LLM-generated outputs. Each successful experiment resulted in five executable simulation models, requiring ten API requests (avg. ~33,800 tokens; cost $0.16). The average execution time of successful experiments was 625 s (≈10 min), excluding runs requiring manual correction.

Run	Model generation					Model adaptation			Model evaluation
	Model result	Error type	KPIs			BT suggestions	CPDs flagged	Adaptation result	Evaluation result
			TH	WIP	SEC				
1	✓	–	27.29	18.32	0.4506	1. Buffers[a] BT1–2 2. AVBs BT1–2 3. CT BT3 4. Parallel press[b]	✓	✓	✓
2	✓	–	27.29	18.32	0.4506	1. Buffer BT1 2. Buffer BT2 3. CT BT1	✓	✓	✓
3	Manual correction	Parametric	27.29	18.32	0.4506	1. Buffers BT1–2 2. CTs BT1–2 3. AVB BT3	✓	✓	✓
4	Manual correction	Structural	27.32	18.21	0.4498	1. Buffers BT1 2. Buffer BT2 3. CT BT3	✓	✓	✓
5	✓	–	27.25	18.28	0.4506	1. Buffers BT1–2 2. MTTR BT1–2 3. CT BT3	✓	✓	✓
6	✓	–	27.29	18.32	0.4506	1. Buffer BT1 2. Buffer BT2 3. CTs BT1–2	✓	✓	✓
7	✓	–	27.26	18.22	0.4510	1. Buffers BT1–2 2. CTs BT1–2 3. Defect BT3	✓	✓	✓
8	✓	–	27.26	18.22	0.4510	1. Buffer BT1 2. Buffer BT2 3. Buffer BT3	✓	✓	✓
9	✓	–	27.29	18.32	0.4506	1. Buffer BT1 2. Buffer BT2 3. CT BT1	✓	✓	✓
10	Manual correction	Structural	27.29	18.32	0.4506	1. Buffers BT1–2 2. CT BT3 3. Buffer BT3	✓	✓	✓
11	Manual correction	Structural & scheduling	27.29	18.32	0.4506	1. Buffer BT1 2. Buffer BT2 3. AVBs BT1–2	✓	✓	✓

[a] All 'buffer' references indicate input buffer capacities.
[b] All runs included a fourth BT suggestion: 'Add parallel press'; this is only shown for the first run, while only suggestions 1–3 are shown for the remaining runs.

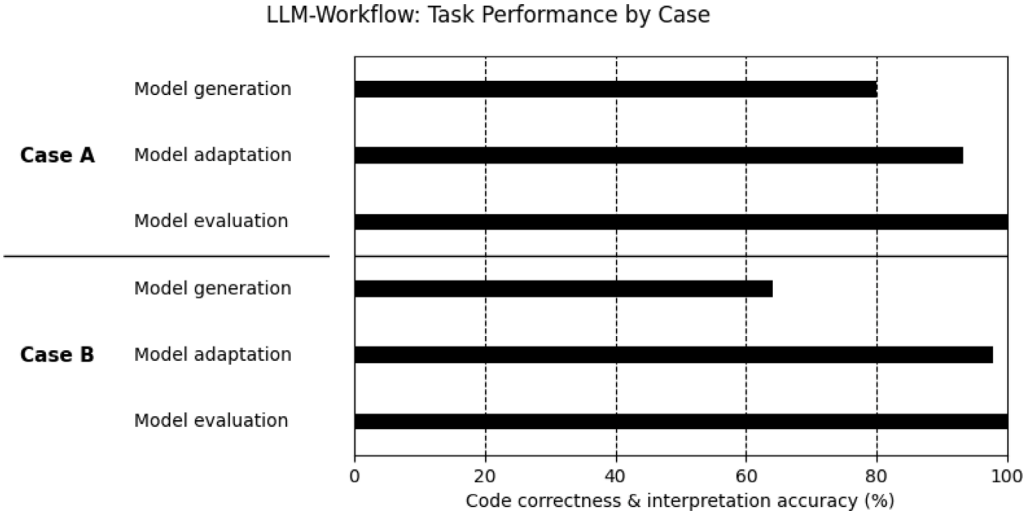

Fig. 18. Summary of code correctness and LLM output accuracy of the LLM-driven model generation, adaptation, and evaluation tasks for Cases A and B.

6.1. Methodological contributions

This work demonstrated that LLMs can generate, adapt, and evaluate executable and valid simulation models of real manufacturing systems when supported by a structured workflow and human-in-the-loop mechanisms. Unlike conventional simulation tools that require coding, parameter configuration, or GUI interactions, the proposed method enables end-to-end automation through natural language input.

Table 8
Relative performance differences (%) between Python-based models generated by the LLM-driven workflow and C++-based models in FACTS Analyzer for Case A for a 30-day simulation horizon with a 1-day warm-up period and 25 replications.

Model	TH [%]	SEC [%]	WIP [%]
Current-state	−3.79	1.96	3.78
V1	−3.77	1.91	6.71
V2	−4.04	1.95	0.71
V3	−4.03	1.96	3.90
Mean	−3.91	1.95	3.77

Table 9
Relative performance differences (%) between Python-based models generated by the LLM-driven workflow and C++-based models in FACTS Analyzer for Case B for a 7-day simulation horizon with a 1-day warm-up period and 10 replications.

Model	TH [%]	SEC [%]	WIP [%]
Current-state	0.85	0.52	1.11
V1	0.63	0.57	−0.46
V2	0.33	0.46	−0.65
V3	0.39	0.82	−1.36
V4	1.77	−0.83	−0.68
Mean	0.79	0.31	−0.41

The LLMs produced the model logic, which the human operator reviewed, controlled, and corrected at designated checkpoints. NLP capabilities were leveraged by translating KPIs into user-friendly, text-based decision support.

The workflow-based approach proved controllable, consistent, and largely reliable, as shown by systematic analyses across multiple experiments in both cases. Achieving this required a combination of Gen-AI techniques: one-shot prompting with a blueprint model enabled the generation of an operational current-state model, as the LLMs alone struggled to generate executable models without guidance. Thus, domain-specific knowledge was fed to the LLM in order to reduce the risk of hallucinations, as LLMs are usually trained on generic data; a problem also encountered by Li et al. [32]. While current LLMs cannot yet guarantee error-free code generation from scratch, they proved more reliable in performing subsequent adaptations once the baseline model was validated. Zero-shot adaptations supported parameter and structural modifications. Evaluator–optimizer loops enhanced robustness by detecting and correcting faulty code. Another measure was prompt refinement, as LLMs are sensitive to unclear instructions, by testing outputs across multiple runs. This reduced errors such as unnecessary wrappers or comments, incorrect library imports, and faulty assumptions. Table 10 summarizes the most commonly observed hallucination mechanisms across workflow stages. The observed hallucinations were based on the specific task at hand, whether it was to produce clean and correct code outputs or concise textual outputs. As different LLMs were intentionally assigned to different workflow tasks based on their capabilities and costs, hallucination mechanisms were analyzed at the task level rather than attributed to individual LLMs.

A key design principle was to combine deterministic, pre-defined code with selectively applied LLM-generated code. This balance ensured sufficient reliability while still allowing the LLM flexibility needed for diverse adaptations. Nevertheless, non-determinism remains inherent in LLM-based generation, and the risk of hallucinations, leading to inconsistent or incorrect code, constitutes the most critical limitation of Gen-AI for industrial use [6]. For example, the builder-agent once invented a nonexistent machine, occasionally introduced incorrect part flows around parallel stations, and the bottleneck-agent at one point provided the adaptor-agent with a logically inconsistent suggestion. These issues illustrate that the LLM does not understand its output; rather, it predicts the most likely sequence of words or code based on its training data and the given instructions, and therefore, due to its probabilistic nature, can generate erroneous results. In addition, the inconsistencies from the builder-agent likely stem from the generality of the blueprint model and the multiple valid ways to represent logic in SimPy.

The risk of hallucinations cannot be eliminated but can be managed through structured workflows. In the present workflow, it was mitigated by careful design and iterative refinement of the blueprint model and prompts. Further, human supervision remained crucial. Operator checkpoints were included to validate models, intervene when inconsistencies are detected, and provide additional improvement suggestions when the LLM overlooked context-specific constraints. This principle of human-in-the-loop aligns with recent best practices for building effective agentic workflows [13]. Emerging evaluation frameworks such as Ragas [47] and LLM-as-a-Judge [48], which systematically measure the relevance and correctness of LLM outputs, may further increase testing capabilities and build trust to ensure reliability before production deployments.

Despite these measures, complete reproducibility cannot be achieved, as LLMs may produce slight variations across executions. This underscores the need for continuous oversight and for defining acceptable tolerance ranges when comparing simulation outcomes. In practice, this reflects a broader trade-off between the creativity that LLMs enable and the reliability required in industrial applications. However, with the expected development of more capable or domain-specialized LLMs, such as those fine-tuned on SimPy models, output consistency and contextual reasoning are expected to improve. This could reduce the need for extensive if–else logic or static templates used in earlier ASMG research [26]. An overview of the workflow including the embedded measures against output inconsistencies is provided in Fig. 19.

6.2. Insights from testing across complexity levels

Beyond demonstrating feasibility, the workflow was tested across a variety of setups to evaluate robustness under different conditions. This iterative testing was essential to ensure that the framework could accommodate diverse production processes and adaptation types. Three dimensions of complexity were explored: (i) different LLMs and workflow configurations (single-agent vs. evaluator–optimizer loop); (ii) varying task complexities, from simple parameter adjustments to structural changes; and (iii) different levels of modeling fidelity, representing increasing degrees of real-world detail (from Case A to Case B). These layers are illustrated in Fig. 20.

One key learning was the role of model selection in balancing efficiency, capability, and cost. The *GPT-5.1* model was chosen for generating the current-state model due to its state-of-the-art coding capabilities, which were expected to yield more reliable code with fewer manual corrections. The more lightweight *GPT-5-mini* model offered a cost-effective solution for various tasks, including routine parameter modifications and implementing structural changes like the additional parallel station in Case B (adaptor-agent). It also handled the translation from Python-based model code into JavaScript-based code for producing the process flow chart (visualizer-agent). The *GPT-4o* model was used due for generating instructions in JSON due to its ability to support structured output.

The systematic analysis across experiment runs shows that LLM performance decreased as model complexity increased. Case B showed a higher frequency of structural errors, mainly associated with the need to split and merge part flows at the two parallel stations. Once a correct initial model for Case B was generated, however, the adaptor-agent consistently implemented the addition of a third parallel station. This improvement suggestion was supported by iterative prompt refinements that provided more explicit guidance, as well as by the fact that the initial model already contained valid merger–splitter logic.

Table 10
Most commonly observed hallucination mechanisms across workflow stages and corresponding mitigation strategies.

Hallucination mechanism	Primary trigger	Mitigation strategy	Workflow stage
Overly verbose code scaffolding	LLM prior towards explanatory or verbose outputs	Helper functions to remove code wrappers and explicit comment instructions, e.g., "Only answer with the code."	Generation/adaptation
Overly verbose textual suggestions	LLM prior towards explanatory or verbose outputs	Explicit output instructions including a worked example, e.g., "These should be short and concise statements, e.g. Increase the buffer size of X to 40."	Adaptation suggestion
Inconsistent part routing logic (e.g., at parallel stations)	Unclear routing instructions	Prompt refinement and clearer routing instructions in blueprint, e.g., "#Merge outputs of parallel machines into buffer3."	Generation/adaptation
Erroneous code (e.g., defining production hours or part generation)	Reuse of generic simulation code patterns without domain constraints	Blueprint refinement and evaluator–optimizer workflows	Generation

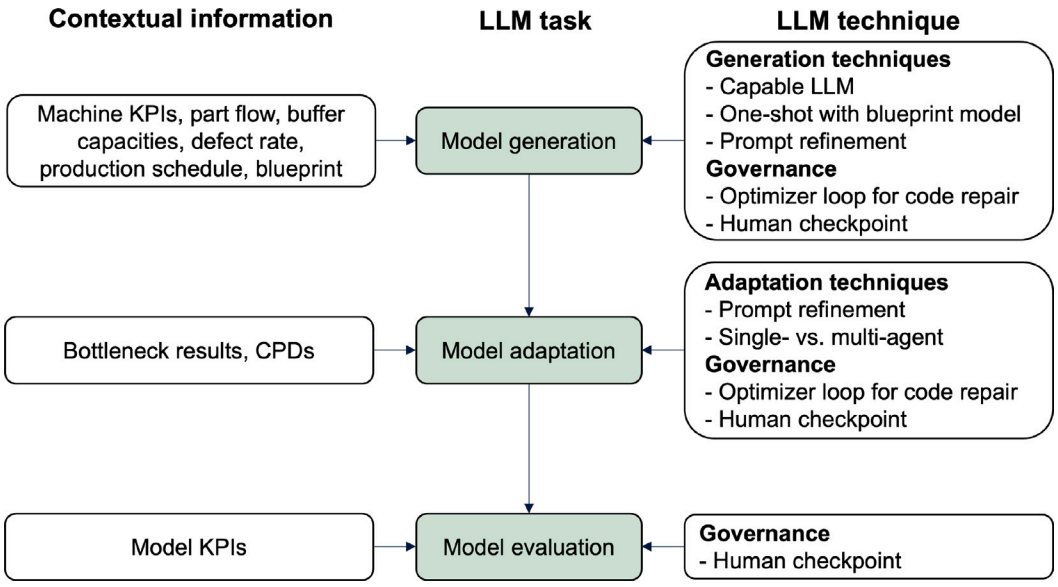

Fig. 19. High-level summary of how contextual information informs LLM tasks and how LLM techniques and governance mechanisms mitigate risks such as hallucinated code and misaligned adaptations.

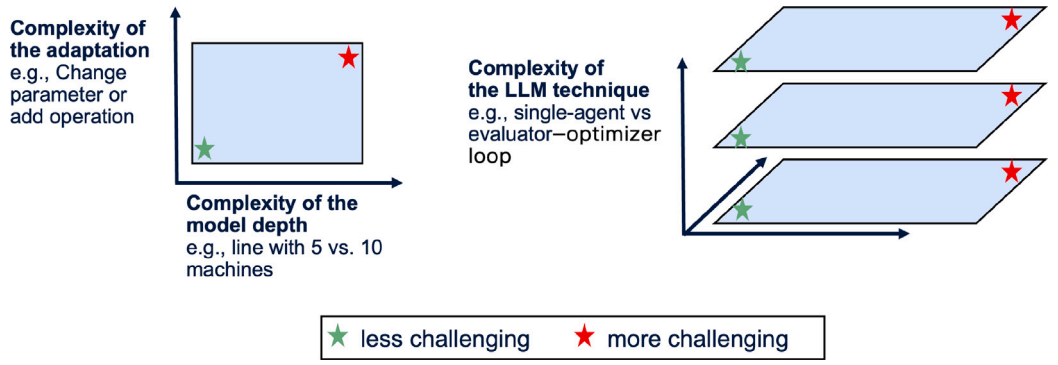

Fig. 20. Complexity levels tested and implemented into the LLM workflow.
Source: Adapted from Gegenmantel [43].

Together, these elements made the adaptation stage more robust to structural complexity than the initial model-building stage. This also highlights a practical implication for workflow design: clearer and more targeted instructions during the initial model-building phase may help reduce structural errors upstream. In contrast, Case A revealed a different error type: in one experiment, the model-builder introduced a parallel station that appeared in the blueprint examples but was not part of the assigned task, suggesting that insufficiently specific instructions can lead the agent to overgeneralize.

Overall, these experiments highlight that Gen-AI workflows are not one-size-fits-all. Instead, model and configuration choices should be guided by task complexity and resource considerations. This flexibility is valuable in industrial settings, where use cases range across different complexity levels.

6.3. Industrial relevance

In practice, manual modeling is constrained by the engineers' time, resources, and expertise. By combining the operator expertise with the LLM, a wider range of solutions can be explored that time constraints would otherwise not permit. This study demonstrated how an LLM could correctly identify and adapt case-specific features, such as the defect-rate parameter introduced only in Case B, which was modified in one experiment. Thus, even though the proposed workflow introduces uncertainty to simulation modeling, it also broadens the exploratory capacity of modeling.

Importantly, the results also indicate that Gen-AI can lower the expertise required for simulation modeling while still achieving performance comparable to commercial simulation software in key metrics such as productivity, energy efficiency, and inventory. The moderate discrepancies reported in Section 5.6, especially for inventory, could be attributed to the underlying failure and repair times of operations, rather than to limitations of the Gen-AI approach itself. More broadly, LLMs show promise for generating tailored adaptations without requiring all process logic to be hard-coded, thereby reducing programming effort and the risk of overlooking fringe cases.

By combining the LLM's 'creative' outputs with operator expertise, new solutions can be considered that human intuition alone may overlook. This potential is particularly valuable for conceptual 'greenfield' models, where higher degrees of freedom allow the LLM to suggest unconventional changes that may inspire human operators. For industrial 'brownfield' cases like in this study, however, strict fidelity to the real production system is required, and excessive flexibility (e.g., inserting infinite buffer capacities) becomes problematic. This underscores the importance of constraining LLM outputs through blueprints, validation mechanisms, and human oversight. For more explorative 'greenfield' cases, settings like LLM temperature, which sets the allowed output 'creativity' of the model, can become a more important parameter.

Nonetheless, all LLM-generated suggestions must be validated against process constraints that remain unknown to the LLM, including layout, quality, and cost requirements. In this study, the workflow explicitly included CPD checks, but integration of additional unstructured data sources (e.g., supplier documents, technical specifications) could strengthen this process further. Overall, the findings indicate that LLM-driven simulation modeling can meaningfully accelerate scenario generation and broaden the space for solution exploration, provided that the results are built into a robust validation and decision-support framework.

6.4. Limitations and future research

The cases tested in this study differed in complexity but remained relatively simple: a sequential line (Case A) and a larger system combining sequential and parallel stations (Case B). Such models can typically be constructed efficiently by simulation experts, whereas substantially more effort is required for systems with complex part flows, multiple product variants, and interdependent processes. Consequently, the generalizability of the proposed approach to more complex production scenarios remains to be validated. Extending the evaluation to such scenarios therefore represents an important direction for future research. In addition, future work should asses whether LLM-driven simulation modeling reduces overall modeling time and supports faster decision-making. Comparative studies against manual modeling practices and established ASMG approaches would be particularly valuable.

In this study, an evaluator–optimizer approach was used, in which one agent was responsible for generating and adapting the simulation model and another agent inspected and corrected the code. This setup showed for cases A and B that it can successfully generate and improve DES models. However, limitations in output consistency were observed especially when generating the more complex Case B (Fig. 18). With higher task and system complexity, more expansive multi-agent workflows may become beneficial. There, an orchestrator-agent divides the task into sub-tasks managed by other worker-agents, which sequentially implement the sub-tasks by performing code adjustments. Although such multi-agent configurations are already implemented in the workflow, they were not systematically evaluated in this study. Investigating their potential benefits and trade-offs, including increased computational and economic cost, represents an important direction for future research, consistent with recent findings in the literature [49].

Future research should systematically investigate the sensitivity of LLM-driven simulation workflows to hyperparameter settings (e.g., temperature and sampling controls), ideally using controlled task-level benchmarks in which multiple models are evaluated under identical conditions. In parallel, recent advances in agentic AI technologies (such as ChatGPT Agent [50] and tool-integration frameworks such as MCP [14]) indicate that increasingly autonomous modeling workflows may become feasible. Evaluating and benchmarking such autonomous systems therefore constitutes a promising direction for future research.

While this study demonstrated successful applications using Python-based models, early experiments revealed limitations with generating C++/XML code used in commercial tools such as FACTS Analyzer. These shortcomings likely reflect biases in public training data such as from Stack Overflow and GitHub, which contain substantially more Python than C++ examples. Future work could address this gap through fine-tuned models specialized in industrial simulation code or through translators bridging Python and C++/XML. Such developments would facilitate closer integration with commercial platforms and expand the applicability of LLM-driven workflows.

Another limitation concerns input data ingestion. In this study, certain information, such as buffer capacities and process logic, had to be added manually. In industry, data preparation often accounts for a large share of simulation modeling effort [25]. Automating these steps through advanced pre-processing could further reduce effort and improve accuracy. Automated data validation routines could additionally ensure data sufficiency before model generation begins. Integrating design of experiments (DOE) or optimization modules could extend the workflow's ability to explore broader search spaces of solutions.

Finally, broader adoption will require attention to human factors. Simulation engineers will increasingly shift from manual coding towards roles of supervision and orchestration. This transition requires developing 'LLM literacy', i.e., skills to detect, diagnose, and correct errors in generated outputs. While this study showed that operators remain indispensable, more advanced human–AI collaboration frameworks could distribute tasks more effectively and strengthen reliability.

7. Conclusion

This study demonstrates the potential of large language models automating discrete-event simulation modeling in manufacturing. Building on real production cases, the proposed framework shows that LLMs can generate, adapt, and evaluate simulation models, with human operator checkpoints. While challenges such as output consistency and reproducibility remain, the approach successfully created valid simulation models, enabled automated scenario exploration, and provided decision-support for productivity, energy efficiency, and inventory trade-offs. The human-in-the-loop mechanism helps control and refine the LLM outputs where necessary, and expand exploration through adding human expertise. Systematic analyses for two case studies confirmed the effectiveness of the framework.

The novelty of this work lies in delivering the first end-to-end, empirically validated LLM-driven workflow for DES in a manufacturing context, extending beyond prior proof-of-concepts. The results highlight how different Gen-AI techniques can be combined to develop effective and reliable simulation models, with human oversight embedded throughout the process. More broadly, the framework illustrates

how Gen-AI can reduce the reliance on scarce simulation expertise required for existing tools and open simulation modeling to a wider range of engineers. Future research should explore comparisons to other simulation automation approaches, test the framework on more complex production scenarios, and test autonomous AI systems. With continued refinement, LLM-workflows can make simulation a more accessible, scalable, and reliable tool for industrial decision-making.

CRediT authorship contribution statement

Thomas Schmitt: Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Nils Gegenmantel:** Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation. **Matías Urenda Moris:** Writing – review & editing, Supervision, Software, Conceptualization. **Pär Mårtensson:** Writing – review & editing, Validation, Resources. **Kaveh Amouzgar:** Writing – review & editing, Writing – original draft, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Thomas Schmitt reports financial support was provided by Scania AB. Nils Gegenmantel reports financial support was provided by Scania AB. Pä'r Mårtensson reports financial support was provided by Scania AB. Thomas Schmitt reports a relationship with Scania AB that includes: employment. Pär Mårtensson reports a relationship with Scania AB that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This study was partially funded by Sweden's Innovation Agency (Vinnova) through the AI-COMPETE project (grant number 2025-01066).

Appendix A. Excerpt of the blueprint model

```
env = simpy.Environment()

# Machine topology example (serial and parallel)
# M1 -> M2 -> [ M3 || M4 ] -> M5
buffer1 = DelayBuffer(env, cap=2, delay=10)
buffer2 = DelayBuffer(env, cap=2, delay=10)
buffer3 = DelayBuffer(env, cap=2, delay=10)

raw_input= simpy.Store(env, capacity=1000)
sink = simpy.Store(env, capacity=100000)
defects = simpy.Store(env, capacity=100000)

# Helper stores for routing in parallel section
branch1_out = simpy.Store(env, capacity=2)
branch2_out = simpy.Store(env, capacity=2)

M1 = Machine(env, 'M1', input_buffer=raw_input,
     output_buffer=buffer1, process_time=5,
     availability=97.79, mttr=74, working_power
     =kwh_per_sec(1.28), waiting_power=
     kwh_per_sec(1.25),)

M2 = Machine(env, 'M2', input_buffer=buffer1,
     output_buffer=buffer2, process_time=20,
     availability=95.0, mttr=100, working_power
     =kwh_per_sec(1.28), waiting_power=
     kwh_per_sec(1.25),)

M3parallel = Machine(env, 'M3parallel',
     input_buffer=buffer2, output_buffer=
     branch1_out, process_time=15, availability
     =90.0, mttr=80, working_power=kwh_per_sec
     (1.28), waiting_power=kwh_per_sec(1.25),)

M4parallel = Machine(env, 'M4parallel',
     input_buffer=buffer2, output_buffer=
     branch2_out, process_time=15, availability
     =90.0, mttr=80, working_power=kwh_per_sec
     (1.28), waiting_power=kwh_per_sec(1.25),)

# Merge outputs of parallel machines into buffer3
merger(env, branch1_out, branch2_out, buffer3)

M5 = Machine(env, 'M5', input_buffer=buffer3,
     output_buffer=sink, process_time=25,
     availability=92.0, mttr=90, working_power=
     kwh_per_sec(1.28), waiting_power=
     kwh_per_sec(1.25), defect_rate=0.089,
     defect_sink=defects,)

machines_list = [M1, M2, M3parallel, M4parallel,
     M5]

# Start part generation.
env.process(part_generator(env, raw_input))

# Run the model to reach steady-state
env.run(until=warmup)
```

Listing 1: A cleaned excerpt of the Python-based blueprint model, which is used as an input for one-shot implementation using the builder-agent.

Appendix B. Examples of LLM calls

```
def _builder(self, blueprint_code,
     stations_table_md, sequence_text, buffers,
     defects, manual_note, sim_time,
     warmup_seconds,
   model = "gpt-5.1"):
   prompt = (
       "Adapt the Python code to represent the
            following production line. "
       "Parallel processes split the stream of
            objects evenly. "
       "Ensure that the capacities of the buffers
            (raw and normal) match the input, and
            that all (helper) buffers have a
            defined capacity.\n\n"
       f"```python\n{blueprint_code}\n```\n\n"
       f"Stations table:\n\n{stations_table_md
            }\n\n"
       f"Buffer list with their capacity and
            process time:\n\n{buffers}\n\n"
       f"Direct-follow relationships:\n\n{
            sequence_text}\n\n"
       f"Defects:\n\n{defects}\n\n"
       f"{manual_note}\n\n"
       f"Simulation time: {sim_time}\n\n"
       f"Warm-up period: {warmup_seconds}\n\n"
       "Only answer with the code.")
   resp = self.client.chat.completions.create(
       model=model, messages=[{"role": "user", "
            content": prompt}])
   try:
       return resp.choices[0].message.content
```

```
        except Exception as e:
            raise RuntimeError(f"No completion was
                returned: {e}")
```

Listing 2: API call and prompt for the builder-agent (OpenAI's GPT-5.1), which generates the initial simulation model.

```
def _modify_code(self, original_code, instruction,
                 model: str = "gpt-5-mini"):
    prompt = (
        f"Here is my Python
            code:\n\n```python\n{original_code}\n```\n\n"
        f"Modify it according to this
            instruction:\n\n{instruction}\n"
        "Only answer with the code.")
    resp = self.client.chat.completions.create(model=model,
        messages=[{"role": "user", "content": prompt}])
    return resp.choices[0].message.content
```

Listing 3: API call and prompt for the adaptor-agent (OpenAI's GPT-5-mini), which adapts the initial simulation model.

Appendix C. Excerpt of the generated simulation model for Case A

```
    buffer1 = DelayBuffer(env, cap=5, delay=10) #
        PostMachine1
    buffer2 = DelayBuffer(env, cap=5, delay=10) #
        PostMachine2
    buffer3 = DelayBuffer(env, cap=5, delay=10) #
        PostMachine3

    raw_input= simpy.Store(env, capacity=1000) #
        bounded raw buffer
    sink = simpy.Store(env, capacity=100000) #
        final sink
    defects = simpy.Store(env, capacity=100000) #
        defect sink

    # SERIAL TOPOLOGY:
    # raw_input-> SV36262 -> buffer1 -> SV32173 ->
        buffer2
    # buffer2 -> SV91991 -> buffer3 -> SV91989 -> sink

    SV36262 = Machine(env, "SV36262",
        input_buffer=raw_input,
        output_buffer=buffer1, process_time=320,
        availability=85.0, mttr=1409,
        working_power=kwh_per_sec(50),
        waiting_power=kwh_per_sec(10))

    SV32173 = Machine(env, "SV32173",
        input_buffer=buffer1,
        output_buffer=buffer2, process_time=290,
        availability=85.0, mttr=718,
        working_power=kwh_per_sec(50),
        waiting_power=kwh_per_sec(10))

    SV91991 = Machine(env, "SV91991",
        input_buffer=buffer2,
        output_buffer=buffer3, process_time=134,
        availability=98.0, mttr=6717,
        working_power=kwh_per_sec(50),
        waiting_power=kwh_per_sec(10))

    SV91989 = Machine(env, "SV91989",
        input_buffer=buffer3, output_buffer=sink,
        process_time=41, availability=99.0,
        mttr=671, working_power=kwh_per_sec(50),
        waiting_power=kwh_per_sec(10))
```

Listing 4: An excerpt of the generated current-state simulation model of Case A during experiment #1.

Appendix D. Excerpt of the generated simulation model for Case B

```
    raw_input= simpy.Store(env, capacity=1000)
    sink = simpy.Store(env, capacity=100000)
    defects = simpy.Store(env, capacity=100000)

    post_loading_buffer = DelayBuffer(env, cap=2,
        delay=10)
    post_conveyor_buffer = DelayBuffer(env, cap=2,
        delay=10)
    post_washing_buffer = DelayBuffer(env, cap=2,
        delay=10)
    pre_press1_buffer = DelayBuffer(env, cap=3,
        delay=32)
    pre_press2_buffer = DelayBuffer(env, cap=3,
        delay=32)
    post_presses_buffer = DelayBuffer(env, cap=3,
        delay=32)

    # Helper stores for routing from parallel presses
    press1_out = simpy.Store(env, capacity=3)
    press2_out = simpy.Store(env, capacity=3)

    loading_robot = Machine(env, "Loading robot",
        input_buffer=raw_input,
        output_buffer=post_loading_buffer,
        process_time=12.0, availability=90.49,
        mttr=68.0,
        working_power=kwh_per_sec(0.72),
        waiting_power=kwh_per_sec(0.25))

    conveyor_belt = Machine(env, "Conveyor belt",
        input_buffer=post_loading_buffer,
        output_buffer=post_conveyor_buffer,
        process_time=6.0, availability=100.0,
        mttr=1.0, working_power=kwh_per_sec(0.0),
        waiting_power=kwh_per_sec(0.0))

    washing_machine = Machine(env, "Washing
        machine",
        input_buffer=post_conveyor_buffer,
        output_buffer=post_washing_buffer,
        process_time=14.0, availability=80.89,
        mttr=269.0,
        working_power=kwh_per_sec(35.24),
        waiting_power=kwh_per_sec(4.28))

    # For Hantering cell, we need an intermediate
        helper buffer because we want a splitter to
        distribute parts evenly to PrePress1Buffer
        and PrePress2Buffer.
    hantering_out = simpy.Store(env, capacity=6) #
        defined capacity >= sum of pre-press caps
```

```
    hantering_cell = Machine(env, "Hantering cell",
        input_buffer=post_washing_buffer,
        output_buffer=hantering_out,
        process_time=25.0, availability=97.79,
        mttr=74.0,
        working_power=kwh_per_sec(0.74),
        waiting_power=kwh_per_sec(0.50))

    # Split stream from Hantering cell evenly into
        the two pre-press DelayBuffers
    env.process(splitter(env, hantering_out,
        pre_press1_buffer, pre_press2_buffer))

    presses_cell1 = Machine(env, "Presses cell 1",
        input_buffer=pre_press1_buffer,
        output_buffer=press1_out,
        process_time=175.0, availability=87.79,
        mttr=73.0,
        working_power=kwh_per_sec(1.28),
        waiting_power=kwh_per_sec(1.25))

    presses_cell2 = Machine(env, "Presses cell 2",
        input_buffer=pre_press2_buffer,
        output_buffer=press2_out,
        process_time=176.0, availability=87.69,
        mttr=74.0,
        working_power=kwh_per_sec(1.27),
        waiting_power=kwh_per_sec(1.25))

    # Merge parallel presses into
        PostPress1&Press2Buffer
    merger(env, press1_out, press2_out,
        post_presses_buffer)

    quality_station = Machine(env, "Quality station
        cell", input_buffer=post_presses_buffer,
        output_buffer=sink, process_time=41.0,
        availability=85.87, mttr=66.0,
        working_power=kwh_per_sec(0.84),
        waiting_power=kwh_per_sec(0.58),
        defect_rate=0.089, defect_sink=defects)
```

Listing 5: An excerpt of the generated current-state simulation model of Case B during experiment #7, created by the builder-agent and based on the blueprint code and production context information.

References

[1] Gershwin SB. The future of manufacturing systems engineering. Int J Prod Res 2018;56(1–2):224–37. http://dx.doi.org/10.1080/00207543.2017.1395491.

[2] Herrmann C, Thiede S, Kara S, Hesselbach J. Energy oriented simulation of manufacturing systems – concept and application. CIRP Ann 2011;60(1):45–8. http://dx.doi.org/10.1016/j.cirp.2011.03.127.

[3] Schmitt T, Sergi Olives J, Amouzgar K, Hanson L, Urenda Moris M. Optimizing energy efficiency and productivity in industrial manufacturing: A simulation-based optimization approach with knowledge discovery. J Manuf Syst 2025;82:748–65. http://dx.doi.org/10.1016/J.JMSY.2025.07.008.

[4] Jackson I, Jesus Saenz M, Ivanov D. From natural language to simulations: applying AI to automate simulation modelling of logistics systems. Int J Prod Res 2024;62(4):1434–57. http://dx.doi.org/10.1080/00207543.2023.2276811.

[5] Zhang L, Chen Z, Ford V. Advancing building energy modeling with large language models: Exploration and case studies. Energy Build 2024;323:114788. http://dx.doi.org/10.1016/j.enbuild.2024.114788.

[6] Naveed H, Khan AU, Qiu S, Saqib M, Anwar S, Usman M, Akhtar N, Barnes N, Mian A. A comprehensive overview of large language models. ACM Trans Intell Syst Technol 2025;16(5). http://dx.doi.org/10.1145/3744746.

[7] International Energy Agency. Energy and AI. 2025, https://www.iea.org/reports/energy-and-ai. [Accessed 31 August 2025].

[8] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I. Attention is all you need. 2023, arXiv:1706.03762. URL https://arxiv.org/abs/1706.03762.

[9] Gozalo-Brizuela R, Garrido-Merchán EE. A survey of generative AI applications. J Comput Sci 2023;20(8):801–18. http://dx.doi.org/10.3844/jcssp.2024.801.818.

[10] Kallat F, Pfrommer J, Bessai J, Rehof J, Meyer A. Automatic building of a repository for component-based synthesis of warehouse simulation models. Procedia CIRP 2021;104:1440–5. http://dx.doi.org/10.1016/J.PROCIR.2021.11.243.

[11] Ray PP. ChatGPT: A comprehensive review on background, applications, key challenges, bias, ethics, limitations and future scope. Internet Things Cyber-Phys Syst 2023;3:121–54. http://dx.doi.org/10.1016/J.IOTCPS.2023.04.003.

[12] Jackson I, Ivanov D, Dolgui A, Namdar J. Generative artificial intelligence in supply chain and operations management: a capability-based framework for analysis and implementation. Int J Prod Res 2024;62(17):6120–45. http://dx.doi.org/10.1080/00207543.2024.2309309.

[13] Anthropic. Building effective AI agents. 2024, https://www.anthropic.com/engineering/building-effective-agents. [Accessed 31 August 2025].

[14] Anthropic. Introducing the model context protocol. 2024, https://www.anthropic.com/news/model-context-protocol. [Accessed 31 August 2025].

[15] Mawson VJ, Hughes BR. The development of modelling tools to improve energy efficiency in manufacturing processes and systems. J Manuf Syst 2019;51:95–105. http://dx.doi.org/10.1016/J.JMSY.2019.04.008.

[16] Son YJ, Wysk RA. Automatic simulation model generation for simulation-based, real-time shop floor control. Comput Ind 2001;45(3):291–308. http://dx.doi.org/10.1016/S0166-3615(01)00086-0.

[17] Almeida E, Tilbury D. Automatic logic generation for reconfigurable cell-based manufacturing systems. IFAC Proc Vol 2004;37(18):33–8. http://dx.doi.org/10.1016/S1474-6670(17)30718-8.

[18] Wy J, Jeong S, Kim BI, Park J, Shin J, Yoon H, Lee S. A data-driven generic simulation model for logistics-embedded assembly manufacturing lines. Comput Ind Eng 2011;60(1):138–47. http://dx.doi.org/10.1016/J.CIE.2010.10.011.

[19] Burnett GA, Medeiros DJ, Finke DA, Traband MT. Automating the development of shipyard manufacturing models. In: 2008 winter simul conf. 2008, p. 1761–7. http://dx.doi.org/10.1109/WSC.2008.4736264.

[20] Du J, He Q, Fan X. Automating generation of the assembly line models in aircraft manufacturing simulation. In: Proc IEEE int symp assem manuf ISAM. 2013, p. 155–9. http://dx.doi.org/10.1109/ISAM.2013.6643515.

[21] Lugaresi G, Matta A. Automated digital twins generation for manufacturing systems: A case study. IFAC Pap Online 2021;54(1):749–54. http://dx.doi.org/10.1016/J.IFACOL.2021.08.087.

[22] Garcia FA, Eremeev P, Devriendt H, Naets F, Metin H, Ozer M. Automatic generation of digital-twins in advanced manufacturing: A feasibility study. In: Proc int conf control autom diagnosis ICCAD. 2023, http://dx.doi.org/10.1109/ICCAD57653.2023.10152364.

[23] Sommer M, Stjepandic J, Stobrawa S, von Soden M. Automated generation of digital twin for a built environment using scan and object detection as input for production planning. J Ind Inf Integr 2023;33. http://dx.doi.org/10.1016/J.JII.2023.100462.

[24] dos Santos CH, Montevechi JAB, de Queiroz JA, de Carvalho Miranda R, Leal F. Decision support in productive processes through DES and ABS in the Digital Twin era: a systematic literature review. Int J Prod Res 2022;60(8):2662–81. http://dx.doi.org/10.1080/00207543.2021.1898691.

[25] Skoogh A, Johansson B. Mapping of time-consumption during input data management activities. Simul Notes Eur 2009;19:39–46. http://dx.doi.org/10.11128/sne.19.tn.09937.

[26] Cimino A, Elbasheer M, Longo F, Mirabelli G, Solina V, Veltri P. Automatic simulation models generation in industrial systems: A systematic literature review and outlook towards simulation technology in the industry 5.0. J Manuf Syst 2025;80:859–82. http://dx.doi.org/10.1016/j.jmsy.2025.03.027.

[27] Gabsi AEH. Integrating artificial intelligence in Industry 4.0: insights, challenges, and future prospects–a literature review. Ann Oper Res 2024;1–28. http://dx.doi.org/10.1007/S10479-024-06012-6.

[28] Leng J, Zheng K, Li R, Chen C, Wang B, Liu Q, Chen X, Shen W. AIGC-empowered smart manufacturing: Prospects and challenges. Rob Comput Integr Manuf 2026;97:103076. http://dx.doi.org/10.1016/J.RCIM.2025.103076.

[29] Zhang H, Semujju SD, Wang Z, Lv X, Xu K, Wu L, Jia Y, Wu J, Liang W, Zhuang R, Long Z, Ma R, Ma X. Large scale foundation models for intelligent manufacturing applications: a survey. J Intell Manuf 2025;1–52. http://dx.doi.org/10.1007/S10845-024-02536-7.

[30] Ma Y, Zheng S, Yang Z, Pan H, Hong J. A knowledge-graph enhanced large language model-based fault diagnostic reasoning and maintenance decision support pipeline towards industry 5.0. Int J Prod Res 2025. http://dx.doi.org/10.1080/00207543.2025.2472298.

[31] Hu F, Wang C, Wu X. Generative artificial intelligence-enabled facility layout design paradigm. Appl Sci 2025;15(10):5697. http://dx.doi.org/10.3390/app15105697.

[32] Li X, Nassehi A, Hu SJ, Joung BG, Gao RX. A large manufacturing decision model for human-centric decision-making. CIRP Ann 2025;74(1):621–5. http://dx.doi.org/10.1016/j.cirp.2025.04.017.

[33] Giabbanelli PJ, Padilla JJ, Agrawal A. Broadening access to simulations for end-users via large language models: Challenges and opportunities. In: 2024 winter simul conf. 2024, p. 2535–46. http://dx.doi.org/10.1109/WSC63780.2024.10838754.

[34] Dengler G, Bazan P, German R, Lalbakhsh P, Liebmann A. A conversational human-computer interface for smart energy system simulation environments. In: 2023 winter simul conf. 2023, p. 2978–89. http://dx.doi.org/10.1109/WSC60868.2023.10408707.
[35] Obinwanne U, Feng W. A case study on AI to automate simulation modelling. In: Commun comp inf sci, vol. 2244 CCIS, 2025, p. 179–86. http://dx.doi.org/10.1007/978-3-031-75201-8_13.
[36] Kmiecik M. Creating a genetic algorithm for third-party logistics' warehouse delivery scheduling via a large language model. J Model Manag 2025;20(4):1138–62. http://dx.doi.org/10.1108/JM2-06-2024-0192.
[37] OpenAI. Introducing GPT-5.1 for developers. 2025, https://openai.com/index/gpt-5-1-for-developers/. [Accessed 01 December 2025].
[38] SWE-bench: Software engineering benchmark collection. 2025, https://www.swebench.com/. [Accessed 01 December 2025].
[39] OpenAI. Compare models. 2025, https://platform.openai.com/docs/models/compare. [Accessed 01 December 2025].
[40] OpenAI. Structured model outputs. 2025, https://platform.openai.com/docs/guides/structured-outputs/how-to-use.pdf. [Accessed 01 December 2025].
[41] Process Intelligence Solutions (PIS). PM4py – process mining for python. 2025, https://pypi.org/project/pm4py/. [Accessed 01 December 2025].
[42] Team SimPy. Simpy documentation. 2025, https://simpy.readthedocs.io/en/latest/. [Accessed 28 August 2025].
[43] Gegenmantel N. Automating the path from data to decision: Generating and adapting simulations with Gen-AI. In: IT, no. mIA 25 003, Uppsala University, Department of Information Technology; 2025, p. 49.
[44] Goldratt EM, Cox J. The goal: A process of ongoing improvement. 3rd ed. North River Press; 2004.
[45] Diaz-Elsayed N, Jondral A, Greinacher S, Dornfeld D, Lanza G. Assessment of lean and green strategies by simulation of manufacturing systems in discrete production environments. CIRP Ann 2013;62(1):475–8. http://dx.doi.org/10.1016/J.CIRP.2013.03.066.
[46] Sargent RG. Verification and validation of simulation models. J Simul 2013;7(1):12–24. http://dx.doi.org/10.1057/jos.2012.20.
[47] Ragas. 2025, https://docs.ragas.io/en/stable/. [Accessed 28 August 2025].
[48] Zheng L, Chiang W-L, Sheng Y, Zhuang S, Wu Z, Zhuang Y, Lin Z, Li Z, Li D, Xing EP, Zhang H, Gonzalez JE, Stoica I. Judging LLM-as-a-judge with MT-bench and chatbot arena. 2023, arXiv:2306.05685. URL https://arxiv.org/abs/2306.05685.
[49] Meyerson E, Paolo G, Dailey R, Shahrzad H, Francon O, Hayes CF, Qiu X, Hodjat B, Miikkulainen R. Solving a million-step LLM task with zero errors. 2025, arXiv:2511.09030. URL https://arxiv.org/abs/2511.09030.
[50] OpenAI. Introducing ChatGPT agent: Bridging research and action. 2025, https://openai.com/index/introducing-chatgpt-agent/. [Accessed 05 December 2025].